Introduction aux architectures orientées microservices

MSOA 1 — Partie I : de l'application monolithique aux microservices

Cours complet : notions, méthodes, cas réels et exercices corrigés

Chaque notion est étudiée en 3 temps

QUOI ? La définition, le principe et ce que l'architecture résout.

OÙ / QUAND ? Les contextes et les signaux qui indiquent qu'elle est le bon choix.

COMMENT ? La démarche pas à pas pour l'analyser et l'appliquer.

Contenu : N1 Motivations des microservices · N2 Architecture monolithique · N3 Scalabilité verticale et horizontale · N4 Monolithe modulaire · N5 Architecture microservices · Problèmes réels (LAB 1) · Banque de 30 exercices corrigés.

Master Big Data & Data Science — d'après le cours du Pr. Zouheir Banou

Section 0

Table des matières

Ouverture — Avant de commencer 5

1. Objectifs du cours 5
2. Un problème réel 6
3. Introduction 6

Notion 1 — Les motivations des microservices 8

4. La règle 8
5. Explication de la règle 9
6. Le problème que la règle résout 10
7. À quoi elle sert 11
8. D'où vient la règle 12
9. Où l'utiliser 12
10. Quand l'utiliser 13
11. Comment l'utiliser 14
12. Exercices d'application corrigés 15
13. Exercices d'entraînement 17
14. Exercices de difficulté croissante 18

Notion 2 — L'architecture monolithique 20

4. La règle 20
5. Explication de la règle 21
6. Le problème que la règle résout 22
7. À quoi elle sert 23
8. D'où vient la règle 23
9. Où l'utiliser 24
10. Quand l'utiliser 25
11. Comment l'utiliser 25
12. Exercices d'application corrigés 26
13. Exercices d'entraînement 29
14. Exercices de difficulté croissante 30

Notion 3 — La scalabilité : verticale et horizontale 31

4. La règle 31
5. Explication de la règle 32
6. Le problème que la règle résout 33

7. À quoi elle sert 34
8. D'où vient la règle 34
9. Où l'utiliser 35
10. Quand l'utiliser 35
11. Comment l'utiliser 36
12. Exercices d'application corrigés 37
13. Exercices d'entraînement 39
14. Exercices de difficulté croissante 39

Notion 4 — Le monolithe modulaire 41
4. La règle 41
5. Explication de la règle 41
6. Le problème que la règle résout 43
7. À quoi elle sert 44
8. D'où vient la règle 44
9. Où l'utiliser 45
10. Quand l'utiliser 45
11. Comment l'utiliser 46
12. Exercices d'application corrigés 47
13. Exercices d'entraînement 50
14. Exercices de difficulté croissante 51

Notion 5 — L'architecture microservices 53
4. La règle 53
5. Explication de la règle 54
6. Le problème que la règle résout 56
7. À quoi elle sert 57
8. D'où vient la règle 57
9. Où l'utiliser 58
10. Quand l'utiliser 59
11. Comment l'utiliser 59
12. Exercices d'application corrigés 61
13. Exercices d'entraînement 63
14. Exercices de difficulté croissante 64

Section 15 — Problèmes réels et solutions détaillées 66
Cas 1 — La librairie de quartier 67
Cas 2 — Le commerçant en croissance 70
Cas 3 — La place de marché nationale 73

Cas 4 — L'entreprise régionale (à défendre) 76

Section 16 — Banque d'exercices 79

Exercices faciles 79

Exercices moyens 80

Exercices difficiles 82

Section 17 — Solutions détaillées de la banque d'exercices 85

Solutions des exercices faciles 85

Solutions des exercices moyens 89

Solutions des exercices difficiles 93

Ouverture

Avant de commencer

1. Objectifs du cours

À la fin de ce cours, vous serez capable de…

- **Expliquer les motivations** qui poussent une entreprise à abandonner une application d'un seul bloc : accès multi-plateformes, hétérogénéité technologique et besoin de scalabilité indépendante.
- **Situer les microservices dans l'histoire** des architectures logicielles : systèmes centralisés (1960s), client-serveur et monolithes (1980s), 3-tiers (1990s), SOA (2000), microservices (2010s), cloud-native (2020s).
- **Définir une architecture monolithique**, citer ses caractéristiques, ses avantages, ses limites, et reconnaître les signes qui montrent qu'un monolithe devient difficile à faire évoluer.
- **Distinguer scalabilité verticale (scale up) et horizontale (scale out)** et les comparer selon l'implémentation, la limite de croissance, le coût et la disponibilité.
- **Décrire le monolithe modulaire** : modules faiblement couplés organisés par capacité métier, frontières claires, communication in-process, une seule unité déployable.
- **Définir l'architecture microservices** et ses six principes (responsabilité unique, couplage faible, flexibilité technologique, déploiement indépendant, scalabilité indépendante, communication par API/messaging).
- **Choisir et défendre une architecture** (monolithe, monolithe modulaire ou microservices) pour un cas réel, en s'appuyant sur des faits du contexte, en écartant les alternatives et en nommant un inconvénient et un déclencheur de révision.

2. Un problème réel

Domaine : e-commerce — « Notre site ne suit plus »

Une enseigne marocaine de vente en ligne a démarré il y a cinq ans avec un **site web** unique : une seule application Java/Spring qui gère le catalogue, les paniers, les commandes, les paiements et les comptes clients, le tout déployé sur un seul serveur et branché sur une seule base de données.

Aujourd'hui, la direction demande trois choses en même temps :

1. **Être présente partout** : lancer une application Android, une application iOS et une application Android TV offrant les mêmes fonctionnalités que le site.
2. **Intégrer l'intelligence artificielle** : un moteur de recommandation de produits que l'équipe data a écrit en **Python**, alors que toute l'application est en Java.
3. **Tenir le Black Friday** : l'an dernier, ce jour-là, la page *catalogue* a reçu dix fois plus de visites que d'habitude ; le serveur a saturé et *tout* le site est tombé, y compris le paiement, qui lui n'était pas surchargé.

Les développeurs constatent en plus qu'une petite correction dans le module des comptes oblige à **retester et redéployer toute l'application**, et que les cinq équipes se gênent en modifiant le même code.

Question : faut-il écrire un backend par application ? Comment brancher un service Python sur une application Java ? Comment renforcer uniquement le catalogue sans payer pour tout le reste ? Et, surtout, **quelle architecture adopter** ?

3. Introduction

Le problème de cette enseigne n'est ni un bug ni un manque de compétences : c'est un problème d'**architecture**. L'application a été pensée comme un seul bloc, ce qui était le bon choix au départ — petite équipe, un seul canal, un seul langage. Mais en grandissant, l'entreprise a changé de besoins : plusieurs interfaces (web, mobile, TV) doivent partager la même logique métier, plusieurs technologies doivent cohabiter, et chaque partie du système subit une charge différente.

Ce cours suit le chemin naturel de cette réflexion. Nous partirons d'abord des **motivations** (N1) : les trois scénarios — multi-plateformes, hétérogénéité technologique et scalabilité indépendante — et l'évolution historique des architectures, des systèmes centralisés jusqu'au cloud-native. Nous étudierons ensuite l'**architecture monolithique** (N2), son fonctionnement, ses forces et ses limites,

puis la notion de **scalabilité** (N3), verticale ou horizontale, qui explique pourquoi un monolithe finit par coûter cher quand une seule de ses parties est sollicitée.

Nous verrons alors une étape intermédiaire souvent oubliée, le **monolithe modulaire** (N4), qui apporte de l'ordre sans multiplier les déploiements, avant d'arriver à l'**architecture microservices** (N5), ses principes et ses conditions de réussite. Le fil conducteur sera toujours le même : il n'existe pas d'architecture « meilleure » dans l'absolu, mais une architecture **adaptée au contexte** — taille de l'application, des équipes, exigences de cohérence et maturité de l'infrastructure. À la fin, les quatre cas du LAB 1 vous demanderont de choisir et de **défendre** une architecture, exactement comme vous devriez le faire pour l'enseigne ci-dessus.

Notion 1

Les motivations des microservices

4. La règle

QUOI ? — Pourquoi les microservices ?

Quand une application grandit en **fonctionnalités**, en **complexité** et en **charge**, trois besoins architecturaux apparaissent :

1. **Accessibilité multi-plateformes** — plusieurs applications (site web, Android, iOS, Android TV) offrent les mêmes fonctionnalités. *Solution* : **un seul backend, plusieurs applications** ; les fonctionnalités métier communes sont centralisées et exposées par des **API** (Application Programming Interfaces).
2. **Hétérogénéité technologique** — un nouveau module doit être écrit dans une autre technologie que le système existant (ex. : IA en Python dans une application Java/Spring Boot). *Solution* : développer ce module comme un **service indépendant** qui expose ses fonctionnalités par une API, appelée par l'application existante via un protocole standard comme **HTTP**.
3. **Scalabilité indépendante** — une seule fonctionnalité subit une forte charge (ex. : le catalogue pendant le Black Friday). *Solution* : diviser l'application en **services indépendants**, chacun responsable d'une fonctionnalité, et n'allouer des ressources supplémentaires **qu'aux services** qui subissent le trafic.

Ces trois besoins motivent une approche dans laquelle une application est organisée en **petits services indépendants et interconnectés** : c'est l'**architecture microservices**.

Cette approche est l'aboutissement d'une évolution historique :

Période	Architecture	Idée principale
1960s	Systèmes centralisés	Applications et données traitées sur des mainframes.
1980s	Client-serveur & monolithes	Le client-serveur se généralise ; les applications monolithiques restent courantes.
1990s	Architecture 3-tiers	Séparation des couches présentation, logique métier et données.

Période	Architecture	Idée principale
2000	SOA (Service-Oriented Architecture)	Services réutilisables et interopérabilité.
2010s	Microservices	Services déployables indépendamment, organisés autour des capacités métier.
2020s	Cloud-native	Conteneurs, orchestration, serverless et déploiement automatisé.

5. Explication de la règle

Ce que dit vraiment la règle

L'idée centrale est simple : **une application qui grandit cesse d'être homogène**. Au début, tout va ensemble — un seul client (le site), un seul langage, une charge à peu près uniforme. Avec la croissance, ces trois « uniformités » se brisent : il y a plusieurs clients, plusieurs technologies, et des parties du système beaucoup plus sollicitées que d'autres. Les trois scénarios ne sont donc pas trois problèmes isolés, mais trois facettes d'un même phénomène : le système doit devenir **plus flexible** que le bloc unique de départ.

Les trois solutions partagent un même geste : **séparer ce qui doit vivre séparément et faire communiquer les parties par des interfaces standard (API)**. Dans le scénario 1, on sépare la logique métier de l'interface utilisateur : un backend, plusieurs fronts. Dans le scénario 2, on sépare un module de la technologie de l'application principale : il tourne dans son propre runtime. Dans le scénario 3, on sépare les fonctionnalités pour qu'elles puissent recevoir chacune les ressources dont elles ont besoin. Pousser ce geste jusqu'au bout — découper l'application en petits services indépendants — donne les microservices.

L'historique montre que ce n'est pas une mode soudaine : depuis les mainframes, l'architecture logicielle ne cesse de **séparer les responsabilités** (client / serveur, puis présentation / métier / données, puis services réutilisables, puis services déployables indépendamment). Les microservices sont l'étape où l'unité de séparation devient la **capacité métier** et où chaque partie peut être **déployée seule** ; le cloud-native fournit ensuite l'outillage (conteneurs, orchestration) qui rend cette séparation exploitable à grande échelle.

Pièges et cas particuliers

- **Scénario 1 ≠ microservices obligatoires.** « Un backend + des API » peut parfaitement être un monolithe qui expose une API. Le scénario 1 interdit la *duplication des backends*, pas le bloc unique.
- **Scénario 3 : la solution n'est pas « un plus gros serveur ».** Augmenter les ressources de toute l'application est précisément ce qui crée le gaspillage décrit dans les défis.
- **SOA et microservices ne sont pas synonymes** : la SOA (2000) insiste sur la réutilisation et l'interopérabilité ; les microservices (2010s) insistent sur le déploiement indépendant et l'organisation par capacité métier.
- **Une motivation n'est pas une décision.** Constater un des trois besoins ne suffit pas à adopter les microservices : le choix final dépend du contexte (notions N2 à N5).

6. Le problème que la règle résout

Les défis de chaque scénario

Scénario	Défis sans la solution
1. Multi-plateformes (un backend par application)	**Duplication de code** : la même logique métier est implémentée plusieurs fois. **Complexité de maintenance** : chaque modification doit être appliquée à plusieurs backends. **Incohérence** : les différentes applications peuvent se comporter différemment. **Coûts plus élevés** : plus d'effort de développement, de test et de maintenance.
2. Hétérogénéité technologique (intégrer Python dans Java)	**Incompatibilité de langage** : Java et Python ont des runtimes et des écosystèmes différents. **Complexité d'intégration** : intégrer directement du Python dans Java exige des mécanismes supplémentaires. **Dépendances de déploiement** : une modification du module IA peut affecter le déploiement de l'application existante. **Maintenabilité** : des composants fortement couplés sont plus difficiles à mettre à jour indépendamment.

Scénario	Défis sans la solution
3. Scalabilité indépendante (faire grossir toute l'application)	**Gaspillage de ressources** : on alloue des ressources à des composants qui n'en ont pas besoin. **Coûts d'infrastructure plus élevés** : ces ressources inutiles augmentent les dépenses. **Goulots d'étranglement** : un composant surchargé peut dégrader la performance de toute l'application. **Flexibilité limitée** : on ne peut pas faire grossir une fonctionnalité seule si elle partage l'unité de déploiement.

En résumé, la règle répond à une question précise : **comment faire évoluer une application qui grandit sans dupliquer, sans coupler des technologies incompatibles et sans payer pour des ressources inutiles ?**

7. À quoi elle sert

L'utilité pratique

- **Lancer un nouveau canal sans réécrire le métier** : une application iOS ou TV n'est plus qu'un nouveau client de l'API existante ; les règles métier sont écrites, testées et corrigées une seule fois, et toutes les applications se comportent pareil.
- **Choisir le bon outil pour chaque besoin** : l'équipe data garde Python, PyTorch et scikit-learn ; l'équipe principale garde Java/Spring Boot. Chacune met à jour son service sans redéployer celui de l'autre.
- **Payer la charge là où elle est** : pendant le Black Friday, on ajoute des ressources au catalogue seulement ; l'authentification, les commandes et les paiements restent au niveau normal et ne sont plus entraînés dans la chute du catalogue.
- **Argumenter un choix d'architecture** : savoir nommer le scénario et ses défis permet de justifier, devant une équipe ou un client, pourquoi une évolution architecturale est nécessaire.

8. D'où vient la règle

Le raisonnement qui y mène

La règle n'est pas posée a priori : elle est **déduite de trois situations concrètes** par le même raisonnement en trois temps — *approche naïve → défis observés → solution*.

- **Scénario 1.** L'approche naïve consiste à donner à chaque application son propre backend (logique métier, accès aux données, authentification). Comme ces applications offrent les *mêmes* fonctionnalités, chaque règle métier existe en quatre exemplaires : d'où duplication, maintenance multipliée, incohérence et coûts. La seule façon d'éviter les copies est de n'avoir **qu'un exemplaire** de la logique et de le rendre accessible à tous : c'est l'API.
- **Scénario 2.** L'approche naïve consiste à intégrer le code Python dans l'application Java. Or deux runtimes différents ne cohabitent pas naturellement dans un même processus ; il faut des mécanismes d'intégration, et le module IA devient lié au cycle de déploiement de l'application. Si l'on place le module dans son **propre processus** et qu'on communique par HTTP, le langage de chacun n'a plus d'importance : HTTP est un protocole neutre.
- **Scénario 3.** L'approche naïve consiste à faire grossir toute l'application. Mais la charge n'est pas uniforme : des milliers de clients parcourent le catalogue, une petite partie seulement commande. Grossir le tout gaspille, et ne pas grossir laisse le catalogue saturer et entraîner le reste. Pour donner des ressources à **une seule fonctionnalité**, il faut qu'elle soit une **unité séparée** : d'où le découpage en services.

Dans les trois cas, la solution consiste à **séparer une partie et la faire communiquer par API**. Généraliser cette conclusion à toute l'application donne la définition des microservices. L'historique (mainframes → client-serveur → 3-tiers → SOA → microservices → cloud-native) confirme la tendance : chaque étape répond aux limites de la précédente en séparant davantage les responsabilités.

9. Où l'utiliser

OÙ ? — Les contextes d'application

- **Produits numériques multicanaux** (banque en ligne, e-commerce, streaming, transport). Une même offre est servie sur le web, le mobile et parfois la télévision ou des objets connectés. C'est le terrain du scénario 1 : le métier (comptes, catalogue, abonnements) doit être unique et exposé par API, sinon chaque nouvel écran multiplie le code à maintenir.

- **Systèmes qui intègrent de la donnée et de l'IA** (recommandation, détection de fraude, prévision). Ces modules sont presque toujours écrits en Python, avec des bibliothèques comme PyTorch ou scikit-learn, alors que le cœur du système d'information est souvent en Java ou .NET. Le scénario 2 s'applique : on encapsule le modèle dans un service appelé par HTTP.
- **Plateformes à pics de charge** (soldes, Black Friday, inscriptions universitaires, billetterie, résultats d'examens). Une seule fonctionnalité (catalogue, consultation, réservation) concentre le trafic pendant une période courte. Le scénario 3 s'applique : il faut pouvoir renforcer cette seule partie.
- **Exercices de cours et d'examen** : questions du type « identifiez la motivation », « citez les défis », « proposez une solution », « associez une période à une architecture » ; et, au LAB 1, l'analyse d'un cas d'entreprise avant de choisir une architecture.

10. Quand l'utiliser

QUAND ? — Les situations et leurs indices

Situation	Indices dans l'énoncé	Pourquoi c'est ce scénario
Plusieurs interfaces partagent le même métier	« site + application mobile », « mêmes fonctionnalités », « nouvelle plateforme », « chaque appli a son backend »	Le problème vient de la répétition de la logique métier. Tant que chaque client a son backend, toute règle est copiée ; l'indice « mêmes fonctionnalités » signale qu'un backend unique exposé par API suffit.
Une partie doit être écrite dans une autre technologie	« équipe Python », « IA », « bibliothèque disponible seulement en… », « runtime différent », « l'application existante est en Java »	Le problème vient de la cohabitation de deux écosystèmes. L'indice est la présence de deux langages ou frameworks pour un même système ; la séparation en service avec API HTTP rend le langage indifférent.

Situation	Indices dans l'énoncé	Pourquoi c'est ce scénario
La charge est inégale entre fonctionnalités	« pic », « Black Friday », « seule la page X est saturée », « tout le site tombe », « on paie des serveurs pour rien »	Le problème vient de l'unité de déploiement partagée. L'indice est l'écart entre une fonctionnalité très sollicitée et les autres ; seule une séparation permet de renforcer uniquement la partie chargée.
Question historique	une date (1960s, 1980s, 1990s, 2000, 2010s, 2020s), « mainframe », « 3-tiers », « SOA », « conteneurs », « serverless »	On mobilise la frise : chaque période est associée à une architecture et à une idée clé ; il suffit de retrouver le bon couple.

Contre-indice : une seule interface, une seule technologie et une charge uniforme — aucune des trois motivations n'est présente ; rien ne pousse alors à séparer l'application.

11. Comment l'utiliser

COMMENT ? — Analyser les motivations d'un cas en 5 étapes

1. **Relever les faits du contexte** sur trois axes : les *clients* (quelles applications ?), les *technologies* (quels langages, quelles équipes ?), la *charge* (quelles fonctionnalités, quel trafic ?). *Pourquoi :* chaque scénario correspond à un axe ; sans faits, on ne peut rien justifier. *Résultat attendu :* trois courtes listes.
2. **Tester chaque scénario avec son indice** : mêmes fonctionnalités sur plusieurs applications → S1 ; module dans une autre technologie → S2 ; charge inégale entre fonctionnalités → S3. *Pourquoi :* un cas peut contenir zéro, un, deux ou trois scénarios. *Résultat :* la liste des scénarios présents.
3. **Décrire l'approche naïve et nommer ses défis** avec le vocabulaire exact (duplication, maintenance, incohérence, coûts / incompatibilité, intégration, dépendances de déploiement, maintenabilité / gaspillage, coûts, goulots, flexibilité). *Pourquoi :* c'est la preuve que le besoin est réel. *Résultat :* pour chaque scénario, 2 à 4 défis reliés aux faits.
4. **Proposer la solution associée** : backend unique + API (S1) ; service indépendant appelé par HTTP (S2) ; services indépendants et ressources ciblées (S3). *Pourquoi :* chaque défi a sa réponse. *Résultat :* une phrase de solution par scénario.

5. **Conclure** : plus il y a de scénarios présents, plus la séparation en services est motivée ; mais la décision finale d'architecture dépend aussi de la taille, des équipes et de l'infrastructure (N2–N5). *Résultat :* une conclusion nuancée.

À retenir

Multi-plateformes, hétérogénéité technologique et scalabilité indépendante : trois besoins, une même réponse — séparer en services indépendants qui communiquent par API.

12. Exercices d'application corrigés

Exercice 1.1 — Une banque sur trois écrans

Une banque possède un site de banque en ligne. Elle veut lancer une application Android et une application iOS offrant la consultation des comptes, les virements et la gestion des bénéficiaires. Le chef de projet propose de confier chaque application à une équipe différente, chacune développant son propre backend. Analysez cette situation.

Solution détaillée

Étape 1 — Faits. Clients : site web, Android, iOS. Technologies : non précisées (pas de signal). Charge : non précisée. Les trois applications offrent les *mêmes* fonctionnalités (comptes, virements, bénéficiaires).

Étape 2 — Scénarios. L'indice « mêmes fonctionnalités sur plusieurs applications » désigne le **scénario 1**. Aucun indice de S2 (une seule technologie implicite) ni de S3 (pas de pic décrit).

Étape 3 — Défis de l'approche naïve. Trois backends signifient que la règle de virement (plafond, vérification du solde) est codée trois fois : *duplication*. Un changement de plafond réglementaire doit être appliqué partout : *complexité de maintenance*. Si une équipe oublie, un virement refusé sur Android peut être accepté sur iOS : *incohérence*, grave pour une banque. Enfin trois équipes développent et testent la même chose : *coûts plus élevés*.

Étape 4 — Solution. Centraliser la logique métier dans **un seul backend** et l'exposer par des **API** ; le site, Android et iOS deviennent des clients de ces API.

Étape 5 — Conclusion. Une seule motivation est présente ; elle impose un backend unique avec API, mais ne suffit pas à elle seule à imposer des microservices.

Exercice 1.2 — Détecter la fraude

Une plateforme de paiement est développée en Java/Spring Boot. L'équipe data a entraîné un modèle de détection de fraude en Python avec scikit-learn. Un développeur propose de « copier le code Python dans le projet Java » et de livrer le tout dans le même déploiement. Analysez la situation et proposez une solution.

Solution détaillée

Étape 1 — Faits. Clients : non précisés. Technologies : Java/Spring Boot pour le système, Python/scikit-learn pour le modèle, deux équipes. Charge : non précisée.

Étape 2 — Scénarios. Deux langages pour un même système : **scénario 2** (hétérogénéité technologique).

Étape 3 — Défis. Java et Python n'ont pas le même runtime ni le même écosystème : on ne « copie » pas du Python dans Java (*incompatibilité de langage*). Il faudrait un mécanisme supplémentaire pour l'exécuter (*complexité d'intégration*). Chaque réentraînement du modèle obligerait à redéployer toute la plateforme de paiement (*dépendances de déploiement*). Les deux parties, fortement couplées, deviendraient difficiles à faire évoluer séparément (*maintenabilité*).

Étape 4 — Solution. Développer la détection de fraude comme un **service Python indépendant** qui expose une API (par ex. « évaluer cette transaction ») ; l'application Java l'appelle via **HTTP**.

Étape 5 — Conclusion. La motivation S2 est clairement présente : séparer ce module en service est justifié, chaque équipe gardant sa technologie et son rythme de déploiement.

Exercice 1.3 — Résultats du baccalauréat

Le portail d'un ministère gère l'inscription, la consultation des notes et le téléchargement des attestations, dans une seule application déployée sur un serveur. Le jour de la publication des résultats, la consultation des notes reçoit 50 fois le trafic habituel ; le serveur sature et l'inscription devient inaccessible. Le service informatique propose d'acheter un serveur cinq fois plus puissant utilisé toute l'année. Analysez.

Solution détaillée

Étape 1 — Faits. Clients : portail web. Technologies : une seule application. Charge : consultation des notes ×50 un jour par an ; inscription et attestations à trafic ordinaire.

Étape 2 — Scénarios. Charge très inégale entre fonctionnalités : **scénario 3**.

Étape 3 — Défis. La consultation surchargée fait tomber l'inscription, qui n'est pas en cause (*goulot d'étranglement*). Le serveur cinq fois plus puissant donne des ressources à l'inscription et aux attestations qui n'en ont pas besoin (*gaspillage*) et coûte toute l'année pour un pic d'un jour (*coûts d'infrastructure*). Comme tout est dans une seule unité, on ne peut pas renforcer la consultation seule (*flexibilité limitée*).

Étape 4 — Solution. Séparer les fonctionnalités en **services indépendants** et allouer des ressources supplémentaires **au seul service de consultation** le jour des résultats.

Étape 5 — Conclusion. La proposition du service informatique traite le symptôme en gaspillant ; la motivation S3 justifie au contraire une séparation permettant une scalabilité ciblée.

13. Exercices d'entraînement

Exercice 1.4

Citez les quatre défis du scénario « multi-plateformes » et expliquez chacun en une phrase.

Exercice 1.5

Associez chaque période à son architecture et à son idée clé : 1960s, 1980s, 1990s, 2000, 2010s, 2020s.

Exercice 1.6

Une application de streaming (Java) veut ajouter un module de sous-titrage automatique développé en Python. Identifiez le scénario, deux défis de l'intégration directe et la solution.

Exercice 1.7

Une billetterie en ligne voit sa page « réservation » saturer à l'ouverture des ventes d'un concert, alors que la gestion des comptes reste calme. Pourquoi faire grossir toute l'application n'est-il pas la bonne réponse ?

14. Exercices de difficulté croissante

Facile Exercice 1.8

Vrai ou faux, en justifiant : « Pour servir un site et une application mobile offrant les mêmes fonctionnalités, il faut obligatoirement deux backends. »

Facile Exercice 1.9

Quel protocole standard permet à une application Java d'appeler un service Python ? Pourquoi le langage de chaque côté n'a-t-il alors plus d'importance ?

Moyen Exercice 1.10

Expliquez la différence entre la SOA (2000) et les microservices (2010s), puis ce qu'apporte l'époque cloud-native (2020s).

Moyen Exercice 1.11

Une application de livraison de repas existe en web et Android, en Java. Elle veut une application iOS et un module d'estimation des délais en Python. Appliquez la méthode en 5 étapes.

Difficile Exercice 1.12

Reprenez le problème réel de la section 2 (enseigne marocaine). Appliquez la méthode complète : identifiez les trois scénarios, reliez chaque défi à un fait de l'énoncé et proposez les solutions. Dites ensuite pourquoi ces constats ne suffisent pas encore à choisir l'architecture finale.

Difficile **Exercice 1.13**

Un collègue affirme : « Nos trois applications partagent déjà une API unique, donc nous faisons des microservices. » Discutez cette affirmation en vous appuyant sur les trois scénarios et sur la définition donnée en section 4.

Notion 2

L'architecture monolithique

4. La règle

QUOI ? — Définition et caractéristiques

Définition. Une **architecture monolithique** est un style d'architecture logicielle dans lequel les fonctionnalités de l'application sont **développées et déployées comme une seule unité unifiée**.

Caractéristiques clés :

- **Unité de déploiement unique** : toute l'application est déployée ensemble.
- **Environnement d'exécution partagé** : les composants s'exécutent en général dans le même processus applicatif.
- **Composants fortement intégrés** : les modules fonctionnels communiquent par des appels directs de méthodes ou de fonctions.
- **Gestion centralisée** : l'application est couramment construite, testée et déployée comme un tout.

Avantages	Limites
Développement simple : tout est développé dans une seule application, ce qui simplifie le développement initial et la mise en place du projet.	**Scalabilité limitée** : les modules ne peuvent pas être mis à l'échelle individuellement lorsqu'ils appartiennent à la même unité de déploiement.
Déploiement facile : l'application entière est empaquetée et déployée comme une seule unité.	**Complexité croissante** : quand l'application grandit, comprendre, modifier et maintenir le code devient plus difficile.
Tests simplifiés : les tests de bout en bout s'effectuent dans un environnement applicatif unifié.	**Dépendances de déploiement** : mettre à jour un seul module exige en général de reconstruire et redéployer toute l'application.
Communication efficace : les composants internes communiquent par appels directs, sans surcoût réseau.	**Propagation des pannes** : une défaillance dans un module peut affecter la disponibilité ou la stabilité de toute l'application.

Quand le choisir : petites applications, petites équipes de développement, projets à un stade précoce, infrastructure limitée, opérations fortement couplées (interactions fréquentes et transactions atomiques).

Signes qu'il devient difficile à gérer : complexité croissante, surcoût de déploiement (même une petite modification exige de tout redéployer), scalabilité limitée, contraintes technologiques (introduire une nouvelle technologie devient compliqué), flexibilité réduite (les équipes doivent coordonner leurs changements dans la même application).

5. Explication de la règle

Ce que dit vraiment la règle

Le mot clé du monolithe est **« un »** : un code, un processus, un paquet, un déploiement. Ce n'est pas une application mal écrite ; c'est un choix qui consiste à garder **toutes les fonctionnalités ensemble**. Les quatre caractéristiques découlent de ce choix : puisqu'il n'y a qu'une unité, on la déploie d'un coup ; puisque tout tourne dans le même processus, les modules s'appellent directement comme on appelle une fonction ; et puisque tout est ensemble, on construit et on teste tout ensemble.

La règle dit ensuite que ce même « un » est **à double tranchant**. Chaque avantage a sa limite jumelle : le déploiement unique est facile, mais il oblige à tout redéployer pour une petite modification ; les appels directs sont rapides, mais ils lient les modules au point qu'une panne se propage ; l'unité unique est simple à lancer, mais on ne peut la faire grossir qu'en bloc. Tant que l'application est petite, les avantages dominent ; quand elle grandit, les limites prennent le dessus.

D'où la conclusion de la règle : le monolithe n'est ni bon ni mauvais **en soi**. Il est le bon choix quand l'application, l'équipe et l'infrastructure sont petites ou quand les opérations doivent rester atomiques, et il devient un frein quand apparaissent les cinq signes de difficulté. Le choix d'architecture est un **compromis** qui dépend des exigences, de la complexité et des ressources disponibles.

Exemple (traitement d'une commande). Dans une application e-commerce monolithique, le client parcourt les produits, passe commande et paie. Le module Commandes appelle directement une méthode du module Catalogue pour vérifier le stock, puis une méthode du module Paiements : tout se passe dans le même processus, sans réseau, et l'ensemble partage la même unité de déploiement.

Comment les limites apparaissent (vu en cours). Au début, l'application compte peu de modules. Avec le temps, on en ajoute un nouveau (par exemple la gestion des utilisateurs) qui ne peut pas être développé à part : il est couplé aux modules existants et crée de nouvelles dépendances. Dès lors, une petite correction dans ce module oblige à **redéployer toute l'application**. Si l'application a du succès et reçoit beaucoup de commandes, seule la partie « commandes » est surchargée, mais les composants ne pouvant pas être mis à l'échelle indépendamment, on doit allouer des machines entières à toute l'application : une grande partie des ressources reste vide. Enfin, si l'on répartit les développeurs en équipes de cinq, chacune responsable d'un module, toutes travaillent dans le même code et doivent **se coordonner** pour livrer l'application, ce qui crée des conflits.

Pièges et cas particuliers

- **« Monolithe » ne veut pas dire « code sans organisation ».** Un monolithe peut être découpé en packages ou en couches ; ce qui le définit est l'*unité de déploiement unique*, pas le désordre du code.
- **Un monolithe peut scaler** : on peut lui ajouter des ressources ou lancer plusieurs copies. Ce qu'il ne peut pas faire ordinairement, c'est mettre à l'échelle *un seul module*.
- **Exposer une API ne fait pas sortir du monolithe** : un monolithe qui sert le web et le mobile via une API reste un monolithe.
- **Une architecture 3-tiers peut être monolithique** : séparer présentation, métier et données en couches ne change rien si la logique métier est déployée comme une seule unité.

6. Le problème que la règle résout

Le problème

La notion de monolithe répond à deux problèmes. Le premier est un **problème de construction** : comment livrer rapidement une application qui fonctionne, avec peu de personnes et peu de moyens ? Le monolithe y répond en supprimant toute la complexité de la distribution — pas de réseau entre composants, pas de déploiements multiples, pas de coordination entre services, des transactions atomiques dans une seule base.

Le second est un **problème de diagnostic** : pourquoi une application qui marchait bien devient-elle lente à faire évoluer, coûteuse à faire grossir et fragile ? La règle donne le vocabulaire pour le dire précisément : scalabilité limitée, complexité croissante, dépendances de déploiement,

propagation des pannes — et les cinq signes qui indiquent que l'application a dépassé le point où la gérer comme une seule unité reste pratique.

7. À quoi elle sert

L'utilité pratique

- **Démarrer vite et à faible coût** : un prototype, un MVP ou l'outil interne d'une petite équipe se développe, se teste et se déploie sans infrastructure distribuée, ce qui permet de valider une idée rapidement.
- **Garantir la cohérence** : quand une opération touche plusieurs modules (commande + stock + paiement) et doit réussir ou échouer en bloc, l'avoir dans une seule application rend la transaction atomique simple.
- **Reconnaître le bon moment pour évoluer** : les cinq signes de difficulté servent de liste de contrôle ; tant qu'ils n'apparaissent pas, inutile de compliquer l'architecture.
- **Défendre un choix** : au LAB 1 et en entreprise, savoir dire « un monolithe suffit ici, et voici pourquoi » est aussi important que savoir proposer des microservices.

8. D'où vient la règle

Le raisonnement qui y mène

Historiquement, le monolithe est la **forme naturelle** d'une application : dans les années 1980, avec le client-serveur, on écrivait une application comme un programme, compilé et installé en un bloc. La définition décrit donc simplement ce qui se passe quand on ne découpe pas : toutes les fonctionnalités dans une seule unité.

Avantages et limites se **déduisent** ensuite de cette définition, sans rien ajouter :

- Une seule unité ⇒ un seul paquet à construire et déployer ⇒ *déploiement facile* ; mais aussi ⇒ toute modification reconstruit et redéploie le tout ⇒ *dépendances de déploiement*.
- Un seul processus ⇒ appels directs en mémoire ⇒ *communication efficace* ; mais aussi ⇒ les modules partagent les ressources du processus ⇒ une erreur grave touche tout ⇒ *propagation des pannes*.
- Un seul code ⇒ un seul projet à ouvrir et à tester ⇒ *développement et tests simples* ; mais aussi ⇒ ce code grossit avec chaque fonctionnalité ⇒ *complexité croissante*.

- Une seule unité de déploiement ⇒ on ne peut ajouter des ressources qu'à l'unité entière ⇒ *scalabilité limitée*.

Les critères de choix et les signes de difficulté sont les deux faces de ce bilan : le monolithe convient tant que la taille rend les limites négligeables, et il devient difficile quand la croissance les rend dominantes. La question qui suit naturellement — « que faire quand l'application dépasse ce point ? » — mène à la solution des slides : diviser l'application en services plus petits et modulaires (notions N4 et N5).

9. Où l'utiliser

OÙ ? — Les contextes d'application

- **Petites applications métier** (site d'une librairie, gestion d'un cabinet, application interne d'une PME). Les fonctionnalités sont peu nombreuses et stables ; un monolithe se comprend en entier et se maintient facilement par une petite équipe.
- **Startups et projets en phase de lancement** (MVP, prototype). L'objectif est de valider un besoin vite ; la simplicité de développement et de déploiement permet d'itérer rapidement sans gérer une infrastructure distribuée.
- **Systèmes à forte cohérence transactionnelle** (caisse, réservation, gestion de stock simple). Les opérations doivent être atomiques ; les coordonner à l'intérieur d'une seule application est beaucoup plus simple qu'entre services.
- **Diagnostic d'applications existantes** : audits, exercices « cette application est-elle monolithique ? », « quels signes montrent qu'elle devient difficile à gérer ? », et l'analyse des cas du LAB 1 (en particulier le cas 1, Bookstore).

10. Quand l'utiliser

QUAND ? — Les situations et leurs indices

Situation	Indices dans l'énoncé	Lecture
Le monolithe est le bon choix	« petite équipe (2–5 devs) », « peu de fonctionnalités », « lancement », « budget / serveur limité », « commande + stock + paiement en une transaction »	Ce sont les cinq critères des slides. Plus ils sont nombreux, plus la simplicité du monolithe l'emporte sur ses limites, qui restent négligeables à cette taille.
Le monolithe devient difficile	« chaque correction oblige à tout redéployer », « personne ne comprend tout le code », « une panne du module X fait tomber le site », « impossible d'ajouter Python », « les équipes se bloquent »	Ce sont les cinq signes de difficulté. Ils indiquent que l'application a grandi au-delà du point où la gérer comme une unité reste pratique : il faut envisager une évolution.
Identifier un monolithe	« un seul fichier déployé (JAR/WAR) », « un seul processus », « appels de méthodes entre modules », « build et tests globaux »	Ce sont les quatre caractéristiques ; si elles sont présentes, l'application est monolithique, même si son code est bien organisé.

11. Comment l'utiliser

COMMENT ? — Évaluer un monolithe en 5 étapes

1. **Vérifier que l'application est (ou serait) un monolithe** avec les 4 caractéristiques : unité de déploiement unique, environnement d'exécution partagé, appels directs, gestion centralisée. *Pourquoi :* on ne raisonne sur les avantages et limites que si la définition s'applique. *Résultat :* oui / non, justifié.
2. **Confronter le contexte aux 5 critères de choix** : taille de l'application, taille de l'équipe, stade du projet, infrastructure, besoin de transactions atomiques. *Pourquoi :* ils mesurent si la simplicité est l'atout principal. *Résultat :* la liste des critères satisfaits.
3. **Rechercher les 5 signes de difficulté** : complexité, surcoût de déploiement, scalabilité, contraintes technologiques, coordination des équipes. *Pourquoi :* ils mesurent si les limites sont devenues dominantes. *Résultat :* la liste des signes présents.

4. **Faire le bilan avantages / limites dans ce contexte** : relier chaque critère ou signe à l'avantage ou la limite qu'il active (ex. « une seule personne en maintenance » → développement simple ; « pics sur le catalogue » → scalabilité limitée). *Pourquoi :* une décision se défend avec des faits, pas avec des généralités. *Résultat :* un tableau court.
5. **Conclure** : « monolithe adapté » si les critères dominent et qu'aucun signe n'est présent ; « monolithe à faire évoluer » si des signes apparaissent. *Résultat :* une décision justifiée et nuancée.

À retenir

Un monolithe, c'est une seule unité développée et déployée : simple et efficace quand tout est petit, il devient un frein quand l'application, les équipes et la charge grandissent.

12. Exercices d'application corrigés

Exercice 2.1 — Le cabinet dentaire

Un développeur indépendant doit réaliser pour un cabinet dentaire une application de gestion des rendez-vous, des fiches patients et de la facturation. Il y a environ 30 utilisateurs, un seul serveur loué, et la prise de rendez-vous doit réserver le créneau et créer la facture provisoire en une seule opération. Un monolithe est-il adapté ?

Solution détaillée

Étape 1 — Définition. L'application serait développée et déployée comme une seule unité, sur un seul serveur, avec les modules rendez-vous, patients et facturation qui s'appellent directement : c'est un monolithe.

Étape 2 — Critères. Petite application (3 fonctionnalités) ✓ ; petite équipe (1 développeur) ✓ ; infrastructure limitée (un serveur) ✓ ; opérations fortement couplées (créneau + facture en une opération atomique) ✓. Stade du projet : nouveau projet ✓.

Étape 3 — Signes de difficulté. Aucun : 30 utilisateurs, pas de pic de charge, une seule technologie, une seule personne (aucune coordination d'équipes).

Étape 4 — Bilan. Le développement simple et le déploiement facile conviennent à un développeur seul ; la communication directe et la transaction unique garantissent la cohérence créneau/facture ; les limites (scalabilité, propagation des pannes) n'ont presque pas d'effet à cette échelle.

Étape 5 — Conclusion. Un monolithe est **adapté** : les cinq critères sont réunis et aucun signe de difficulté n'apparaît.

Exercice 2.2 — Le monolithe qui s'essouffle

Une plateforme de cours en ligne, monolithique, a grandi : 40 développeurs répartis en 6 équipes, 300 000 lignes de code. Corriger une faute dans la page « profil » impose un déploiement complet de 2 heures. Lors d'un bug mémoire dans le module « vidéos », tout le site (y compris les inscriptions) est tombé. Analysez la situation.

Solution détaillée

Étape 1 — Définition. L'énoncé le dit : application monolithique, un déploiement complet, un processus partagé (un bug mémoire d'un module touche tout).

Étape 2 — Critères. Petite application ✗ (300 000 lignes) ; petite équipe ✗ (40 développeurs) ; projet précoce ✗. Les critères du monolithe ne sont plus réunis.

Étape 3 — Signes. *Complexité croissante* (300 000 lignes) ; *surcoût de déploiement* (2 h de déploiement pour une faute de texte) ; *flexibilité réduite* (6 équipes qui modifient la même application doivent se coordonner) ; la panne illustre aussi la limite *propagation des pannes*.

Étape 4 — Bilan. Les avantages initiaux (déploiement unique, appels directs) se sont retournés : le déploiement unique ralentit chaque livraison et le processus partagé laisse la panne des vidéos atteindre les inscriptions.

Étape 5 — Conclusion. Le monolithe est devenu **difficile à gérer** ; il faut envisager de diviser l'application en unités plus petites et modulaires (voir N4 et N5).

Exercice 2.3 — Monolithe ou pas ?

Une application de gestion de bibliothèque est écrite en trois couches (présentation, métier, données) rangées dans trois dossiers. Elle est compilée en un seul fichier WAR déployé sur un serveur Tomcat ; le module « emprunts » appelle la méthode `verifierDisponibilite()` du module « catalogue ». Un étudiant affirme : « Elle a trois couches, ce n'est donc pas un monolithe. » Qu'en pensez-vous ?

Solution détaillée

Étape 1 — Définition. On vérifie les 4 caractéristiques : un seul WAR ⇒ *unité de déploiement unique* ✓ ; un seul Tomcat ⇒ *environnement d'exécution partagé* ✓ ; appel de `verifierDisponibilite()` ⇒ *appels directs* ✓ ; compilée en un fichier ⇒ *gestion centralisée* ✓.

Étape 2 — Critères. Application de bibliothèque : peu de fonctionnalités, a priori petite équipe ; les critères ne sont pas demandés mais semblent réunis.

Étape 3 — Signes. L'énoncé n'en mentionne aucun.

Étape 4 — Bilan. Les couches sont une *organisation du code* ; elles ne changent ni l'unité de déploiement ni le processus partagé. L'erreur de l'étudiant est de confondre organisation interne et architecture de déploiement.

Étape 5 — Conclusion. L'affirmation est **fausse** : l'application est bien un monolithe (en 3 couches), et vu sa taille, c'est un choix adapté.

13. Exercices d'entraînement

Exercice 2.4

Donnez la définition d'une architecture monolithique et ses quatre caractéristiques clés.

Exercice 2.5

Associez chaque avantage du monolithe à la limite qui en est la « face cachée » et expliquez le lien.

Exercice 2.6

Une association veut une application d'inscription à ses activités (2 bénévoles développeurs, un hébergement mutualisé). Appliquez la méthode en 5 étapes.

Exercice 2.7

Dans une application e-commerce monolithique, décrivez le traitement d'une commande (catalogue, commande, paiement) : comment les modules communiquent-ils, et que se passe-t-il si le module paiement plante ?

14. Exercices de difficulté croissante

Facile **Exercice 2.8**

Vrai ou faux, en justifiant : « Un monolithe ne peut pas du tout être mis à l'échelle. »

Facile **Exercice 2.9**

Citez les cinq situations dans lesquelles le monolithe est le bon choix.

Moyen **Exercice 2.10**

Pourquoi les tests de bout en bout sont-ils plus simples dans un monolithe ? Pourquoi la communication y est-elle plus efficace ? Répondez en lien avec la définition.

Moyen **Exercice 2.11**

Une agence de voyages en ligne (monolithe Java) veut ajouter un comparateur de prix écrit en Python, et ses 4 équipes se plaignent de conflits à chaque livraison. Quels signes de difficulté reconnaissez-vous ? Justifiez chacun.

Difficile **Exercice 2.12**

Reprenez l'enseigne de la section 2. Appliquez la méthode d'évaluation du monolithe et montrez que son architecture était adaptée il y a cinq ans mais ne l'est plus aujourd'hui.

Difficile **Exercice 2.13**

Une start-up de 3 développeurs veut démarrer « directement en microservices pour être prête à grandir ». En vous appuyant sur les critères de choix et les avantages du monolithe, construisez un argumentaire contre ce choix, puis indiquez quel signe devrait, plus tard, les faire changer d'avis.

Notion 3

La scalabilité : verticale et horizontale

4. La règle

QUOI ? — Définition et deux façons de faire grossir une application

Définition. La **scalabilité** est la capacité d'un système à absorber une **charge de travail croissante** (plus d'utilisateurs, de requêtes ou de transactions) en lui **allouant des ressources de calcul supplémentaires**.

- **Scalabilité verticale (*scaling up*)** : on **augmente les ressources d'un serveur existant** (plus de CPU, de mémoire, de disque).
- **Scalabilité horizontale (*scaling out*)** : on **ajoute des serveurs ou des instances** de l'application pour répartir la charge. Au lieu d'améliorer un serveur, on déploie **plusieurs copies de la même application monolithique**.

Les deux approches permettent de supporter une charge croissante : la verticale augmente la capacité d'un serveur existant, l'horizontale répartit la charge sur plusieurs serveurs. **Le choix dépend des exigences de performance, des coûts d'infrastructure et de la complexité du système.**

Aspect	Scaling up (vertical)	Scaling out (horizontal)
Mise en œuvre	Simple à mettre en œuvre	Demande une configuration supplémentaire
Scalabilité	Limitée par la capacité du matériel	Extensible en ajoutant des serveurs
Coût	Le matériel puissant peut coûter cher	Peut utiliser plusieurs serveurs abordables
Disponibilité	Point de défaillance unique (SPOF)	La redondance améliore la disponibilité
Gestion	Infrastructure plus simple à gérer	Exige répartition de charge et coordination
Cohérence des données	Gestion de l'état plus simple	L'état partagé exige une coordination

Aspect	Scaling up (vertical)	Scaling out (horizontal)
Allocation des ressources	On améliore le serveur entier	On ajuste la capacité en ajoutant ou retirant des instances

5. Explication de la règle

Ce que dit vraiment la règle

Quand la charge augmente, il n'y a que deux directions possibles : rendre **la machine plus grosse** ou mettre **plus de machines**. Le vocabulaire le dit en image : *up*, on monte en puissance sur place ; *out*, on s'étale sur plusieurs serveurs. Dans les deux cas, on ajoute des ressources de calcul — c'est la définition même de la scalabilité.

La verticale est **simple** : l'application ne change pas, on la remet sur un serveur plus puissant. Mais elle se heurte à un **plafond matériel** (on ne peut pas ajouter de la mémoire à l'infini), le haut de gamme coûte cher, et il n'y a toujours **qu'un seul serveur** : s'il tombe, tout tombe (point de défaillance unique). Comme il n'y a qu'une copie, l'état (sessions, cache, données en mémoire) reste au même endroit, ce qui le rend simple à gérer.

L'horizontale lève le plafond : on ajoute des instances tant qu'il faut, avec des serveurs ordinaires, et si une instance tombe les autres continuent (**redondance**). En échange, il faut un **répartiteur de charge** (*load balancer*) qui distribue les requêtes, et l'état qui était dans une seule mémoire est maintenant réparti sur plusieurs copies : il faut le **coordonner**. La règle n'élit donc pas de gagnant ; elle donne sept critères pour choisir.

Lien avec le monolithe. Les deux méthodes s'appliquent à un monolithe : on peut le placer sur un serveur plus gros, ou lancer plusieurs copies identiques derrière un répartiteur. Mais dans les deux cas, on fait grossir **l'application entière** : chaque copie contient tous les modules, même ceux qui ne sont pas sollicités.

Exemple vu en cours (Massar). Pendant presque toute l'année, peu de personnes consultent la plateforme ; mais le jour de l'annonce des résultats du baccalauréat, tout le pays s'y connecte en même temps. Faire grossir toute l'infrastructure, toute l'année, pour un seul jour serait un gaspillage ; la bonne réponse est de renforcer **uniquement** la partie qui sert les résultats. Un monolithe ne le permet pas : il faut faire grossir l'ensemble.

Exemple vu en cours (module d'IA). Un monolithe contient un module d'intelligence artificielle qui demande des GPU. Comme tout est déployé ensemble, il faut louer un serveur

équipé de GPU pour **toute** l'application, alors que les autres modules (commandes, comptes…) n'en ont pas besoin. Le scaling up de ce serveur est alors à la fois **coûteux et inefficace**.

Pièges et cas particuliers

- **« Horizontal » ne veut pas dire « microservices ».** Déployer 5 copies du même monolithe est déjà de la scalabilité horizontale ; ce qui change avec les microservices, c'est qu'on peut multiplier *un seul service*.
- **Scaling up n'élimine pas le SPOF** : un serveur dix fois plus puissant reste un serveur unique.
- **Plus de serveurs ≠ moins cher automatiquement** : le tableau dit que l'horizontale *peut* utiliser des serveurs abordables, mais elle ajoute un coût de gestion (répartition de charge, coordination de l'état).
- **Copier une application qui garde un état en mémoire** (sessions locales) sans coordination crée des incohérences : un utilisateur servi par une instance puis par une autre « perd » son panier.

6. Le problème que la règle résout

Le problème

Une application conçue pour 1 000 utilisateurs en reçoit soudain 50 000 : les temps de réponse explosent, les requêtes échouent, le site devient indisponible au pire moment (soldes, résultats d'examen, ouverture d'une billetterie). Le problème est de **suivre la croissance de la charge** sans réécrire l'application.

La règle y répond en donnant deux leviers et, surtout, les **critères pour choisir** entre eux : faut-il une solution immédiate et simple (verticale) ou une capacité extensible et tolérante aux pannes (horizontale) ? Elle révèle aussi la limite du monolithe : les deux leviers font grossir l'application entière, ce qui prépare le besoin de **scalabilité indépendante** vu en N1.

7. À quoi elle sert

L'utilité pratique

- **Choisir une stratégie de montée en charge** justifiée par des critères (coût, disponibilité, complexité) plutôt que par habitude.
- **Améliorer la disponibilité** : savoir qu'une seule machine est un point de défaillance unique conduit à prévoir de la redondance pour les services critiques.
- **Estimer les coûts** : comparer le prix d'un gros serveur à celui de plusieurs petits, en tenant compte de la gestion.
- **Comprendre les limites du monolithe** : on peut le faire grossir, mais seulement en bloc, ce qui gaspille des ressources quand un seul module est surchargé — argument clé pour les architectures des notions N4 et N5.

8. D'où vient la règle

Le raisonnement qui y mène

Partons de la définition : absorber plus de charge *en ajoutant des ressources*. Les ressources de calcul vivent sur des serveurs ; il n'y a donc que deux manières d'en ajouter : **dans** le serveur existant, ou **à côté** de lui. Ce sont exactement le scaling up et le scaling out.

Le tableau se **déduit** de cette différence :

- Dans le même serveur ⇒ rien ne change pour l'application ⇒ *simple à mettre en œuvre et à gérer*, *état simple* ; mais ⇒ un serveur a une taille maximale ⇒ *limité par le matériel*, et le haut de gamme est *cher* ; et ⇒ une seule machine ⇒ *point de défaillance unique*.
- À côté du serveur ⇒ on peut en ajouter autant qu'on veut ⇒ *extensible*, avec des machines *abordables*, *ajustable* en ajoutant/retirant des instances ; et ⇒ plusieurs copies ⇒ *redondance* ; mais ⇒ il faut distribuer les requêtes ⇒ *répartition de charge*, et l'état est éclaté ⇒ *coordination*.

Appliquée au monolithe (slide « Scaling a Monolithic Application »), la seule unité de déploiement implique qu'on ne peut copier ou agrandir que l'ensemble : d'où la limite « scalabilité limitée » de N2, et la question qui conduit aux services séparés.

9. Où l'utiliser

OÙ ? — Les contextes d'application

- **Sites à pics de trafic** (e-commerce pendant le Black Friday, billetterie, résultats du baccalauréat). La charge varie fortement ; la capacité d'ajouter puis retirer des instances rend l'horizontale adaptée.
- **Applications internes à charge stable et modérée** (outil de gestion d'une PME). Un serveur plus puissant suffit souvent ; la simplicité de la verticale l'emporte.
- **Services critiques** (paiement, santé, banque). L'indisponibilité coûte cher ; la redondance de l'horizontale évite le point de défaillance unique.
- **Analyse d'architecture** : justifier au LAB 1 pourquoi un monolithe copié plusieurs fois ne résout pas une surcharge localisée (cas 3, National Marketplace) ou pourquoi un serveur suffit (cas 1, Bookstore).

10. Quand l'utiliser

QUAND ? — Les situations et leurs indices

Situation	Indices dans l'énoncé	Lecture
Scaling up (vertical)	« charge modérée et stable », « besoin rapide », « équipe sans compétence d'exploitation », « application qui garde un état en mémoire », « une interruption courte est acceptable »	La simplicité de mise en œuvre et de gestion de l'état l'emporte, tant que le plafond matériel est loin et que le SPOF est tolérable.
Scaling out (horizontal)	« pics imprévisibles », « croissance continue », « le service ne doit jamais s'arrêter », « serveur déjà au maximum », « budget pour des machines ordinaires »	Il faut une capacité extensible, ajustable et redondante ; on accepte la répartition de charge et la coordination de l'état.

Situation	Indices dans l'énoncé	Lecture
Aucun des deux ne suffit seul	« un seul module est surchargé », « on paie pour multiplier tout le monolithe », « les autres modules sont au repos »	Le problème n'est pas la quantité de ressources mais le fait de ne pouvoir faire grossir que l'ensemble : c'est le besoin de scalabilité indépendante (N1, N5).

11. Comment l'utiliser

COMMENT ? — Choisir une stratégie de scalabilité en 5 étapes

1. **Caractériser la charge** : combien d'utilisateurs/requêtes, croissance (stable, continue, en pics), et quelle partie de l'application est sollicitée. *Pourquoi :* la scalabilité répond à une charge croissante ; il faut savoir laquelle. *Résultat :* un profil de charge.
2. **Relever les contraintes** : budget, disponibilité exigée, compétences d'exploitation, état gardé en mémoire, capacité maximale du serveur actuel. *Pourquoi :* ce sont les critères du tableau (coût, disponibilité, gestion, cohérence, scalabilité). *Résultat :* la liste des contraintes.
3. **Évaluer le scaling up** sur les 7 aspects du tableau. *Pourquoi :* c'est l'option la plus simple ; si elle suffit, inutile de compliquer. *Résultat :* points forts / points bloquants (plafond, SPOF, coût).
4. **Évaluer le scaling out** sur les mêmes aspects. *Pourquoi :* comparer à critères égaux. *Résultat :* points forts / ce qu'il faut ajouter (répartiteur de charge, coordination de l'état).
5. **Conclure et vérifier la granularité** : choisir l'approche (ou une combinaison) et se demander si la charge est globale ou concentrée sur un module. *Pourquoi :* pour un monolithe, les deux leviers font grossir le tout. *Résultat :* une stratégie justifiée, avec sa limite éventuelle.

À retenir

Scaling up = un serveur plus puissant : simple, mais plafonné, coûteux et point de défaillance unique. Scaling out = plus d'instances : extensible et redondant, mais il faut répartir la charge et coordonner l'état. Pour un monolithe, les deux font grossir l'application entière.

12. Exercices d'application corrigés

Exercice 3.1 — L'application de la mairie

Une mairie utilise une application monolithique de demandes d'actes (500 utilisateurs par jour, charge régulière). Le serveur est utilisé à 85 % de sa mémoire et ralentit. L'équipe informatique (2 personnes) n'a jamais géré de répartiteur de charge, et une interruption d'une heure la nuit est acceptable. Quelle stratégie recommander ?

Solution détaillée

Étape 1 — Charge. 500 utilisateurs/jour, charge régulière, sans pic ; c'est la mémoire du serveur qui sature.

Étape 2 — Contraintes. Petite équipe sans compétence en répartition de charge ; disponibilité modérée (une heure d'arrêt acceptable) ; le serveur actuel peut encore être amélioré.

Étape 3 — Scaling up. Ajouter de la mémoire est *simple à mettre en œuvre* et *à gérer* ; le plafond matériel est loin pour 500 utilisateurs ; le SPOF reste, mais l'arrêt d'une heure est toléré.

Étape 4 — Scaling out. Il faudrait un répartiteur de charge et coordonner l'état entre copies : une *configuration supplémentaire* que l'équipe ne maîtrise pas, pour un gain inutile à cette échelle.

Étape 5 — Conclusion. Scaling up : augmenter la mémoire du serveur. La charge est globale et stable, donc faire grossir l'ensemble n'est pas un gaspillage.

Exercice 3.2 — Les résultats du concours

Un site publie les résultats d'un concours national : 2 000 visites par jour habituellement, mais 300 000 en une heure le jour de la publication. Le site ne doit pas tomber ce jour-là. Le serveur actuel est déjà le modèle le plus puissant proposé par l'hébergeur. Quelle stratégie recommander ?

Solution détaillée

Étape 1 — Charge. Charge faible en temps normal, *pic* énorme et court (×150) le jour J.

Étape 2 — Contraintes. Forte disponibilité exigée ; serveur déjà au maximum ; il serait coûteux de payer toute l'année pour une capacité utile une heure.

Étape 3 — Scaling up. Impossible : *limité par la capacité du matériel* (déjà au maximum) ; de plus un serveur unique reste un *point de défaillance unique* le jour où il ne faut pas tomber.

Étape 4 — Scaling out. On ajoute des instances le jour J et on les retire ensuite (*capacité ajustable*) ; la *redondance* protège contre la panne d'une instance ; il faut un répartiteur de charge. Les résultats étant en lecture seule, la coordination de l'état est simple.

Étape 5 — Conclusion. Scaling out temporaire, avec répartiteur de charge. Toute l'application est sollicitée par la consultation, donc copier l'ensemble est cohérent.

Exercice 3.3 — Le catalogue surchargé

Un site e-commerce monolithique (modules catalogue, panier, paiement, comptes) subit pendant les soldes une surcharge du seul module catalogue : il reçoit 90 % des requêtes, les autres modules restent peu utilisés. L'équipe a lancé 8 copies du monolithe derrière un répartiteur. Le site tient, mais la facture d'hébergement a quadruplé. Analysez.

Solution détaillée

Étape 1 — Charge. Pic pendant les soldes, *concentré* sur un module (catalogue, 90 % des requêtes).

Étape 2 — Contraintes. Disponibilité importante (soldes) ; coût devenu problématique.

Étape 3 — Scaling up. Un serveur plus gros ferait grossir tout le monolithe, resterait plafonné et serait un SPOF pendant la période la plus critique : moins adapté.

Étape 4 — Scaling out. C'est ce que l'équipe a fait : la redondance et l'extensibilité ont fonctionné (le site tient). Mais chaque copie embarque aussi panier, paiement et comptes, qui sont presque au repos : on paie 8 fois des modules inutiles.

Étape 5 — Conclusion. Le choix horizontal est correct, mais la **granularité** pose problème : un monolithe ne peut grossir qu'en bloc, d'où le *gaspillage de ressources* et les *coûts* (défis de la scalabilité indépendante, N1). Pouvoir multiplier le seul catalogue demanderait de le déployer séparément (N5).

13. Exercices d'entraînement

Exercice 3.4

Donnez la définition de la scalabilité, puis définissez le scaling up et le scaling out en une phrase chacun.

Exercice 3.5

Reproduisez le tableau comparatif scaling up / scaling out sur ses sept aspects.

Exercice 3.6

Une librairie en ligne (300 commandes par jour, charge stable, un développeur) voit son serveur ralentir. Appliquez la méthode en 5 étapes.

Exercice 3.7

Une plateforme de streaming doit rester disponible 24 h/24 et voit son audience doubler chaque année. Appliquez la méthode en 5 étapes.

14. Exercices de difficulté croissante

Facile Exercice 3.8

Vrai ou faux, en justifiant : « Avec un serveur deux fois plus puissant, mon application n'a plus de point de défaillance unique. »

Facile Exercice 3.9

Classez chaque action en « up » ou « out » : (a) passer de 16 à 64 Go de RAM ; (b) lancer 4 copies du monolithe ; (c) remplacer le processeur par un modèle plus rapide ; (d) ajouter une instance pendant les soldes.

Moyen Exercice 3.10

Expliquez pourquoi le scaling out exige un répartiteur de charge et une coordination de l'état, alors que le scaling up n'en a pas besoin. Illustrez avec un panier gardé en mémoire.

Moyen **Exercice 3.11**

Une application de réservation de cinéma connaît des pics le vendredi soir. Comparez les deux approches sur les aspects coût, disponibilité et allocation des ressources, puis choisissez.

Difficile **Exercice 3.12**

Reprenez l'enseigne de la section 2 (catalogue saturé pendant le Black Friday). Appliquez la méthode complète, puis montrez pourquoi ni le scaling up ni le scaling out du monolithe ne règlent entièrement le problème.

Difficile **Exercice 3.13**

Un directeur affirme : « Le scaling out est toujours meilleur, car il n'a pas de limite. » Construisez une réponse nuancée en vous appuyant sur au moins quatre aspects du tableau, et décrivez un cas où le scaling up est le meilleur choix.

Notion 4

Le monolithe modulaire

4. La règle

QUOI ? — Définition et caractéristiques clés

Définition. Un **monolithe modulaire** est une architecture logicielle dans laquelle l'application est organisée en **modules bien définis et faiblement couplés**, tout en restant **une seule unité déployable**.

Caractéristique	Ce qu'elle signifie
Organisation modulaire	Chaque module prend en charge une **capacité métier** précise.
Déploiement unique	Tous les modules sont **construits et déployés ensemble**.
Frontières claires	Les modules interagissent par des **interfaces internes bien définies**.
Communication simple	Les modules communiquent **dans le même processus** d'application.

En pratique. Chaque capacité métier est organisée en un module séparé, avec des responsabilités et des interfaces internes clairement définies. Cependant, tous les modules font partie de la même application et sont **construits, déployés et mis à l'échelle ensemble**.

Quand le choisir : applications de taille moyenne · exigences évolutives · forte cohérence des données · équipes petites à moyennes · déploiement simple · migration future possible.

5. Explication de la règle

Ce que dit vraiment la règle

Entre le monolithe et les microservices, il n'y a pas de « noir et blanc » : il y a du **gris**. Le monolithe modulaire est ce terrain intermédiaire. **Ce qui ne change pas** par rapport au monolithe : un seul code, une seule unité de déploiement, un seul processus. **Ce qui change** : à l'intérieur, le code est découpé en modules selon les **capacités métier** (Commandes, Produits, Paiements, Authentification…), et chaque module garde pour lui son fonctionnement interne.

Le mot clé est **interface**. Si le module Commandes a besoin d'informations sur un produit, il n'importe pas directement les classes internes du module Produits et ne lit pas sa table : il appelle une **interface** que Produits expose, un peu comme une API REST expose des points d'entrée sans montrer le code. Commandes ne sait pas « comment ça se passe dedans » ; il sait seulement ce que Produits *accepte de faire* pour lui. Ces frontières (ce que Commandes peut faire, ce que Produits peut faire) sont fixées **avant** de développer.

Pourquoi c'est important. Sans frontières, les modules s'appellent les uns les autres dans tous les sens. Analogie vue en cours : je demande une information à Ahmed, qui me renvoie vers un autre, qui me renvoie vers un troisième… Quand l'application grandit, on obtient une chaîne d'imports entre 5, 6, 10 modules où personne ne sait plus qui dépend de qui. Les interfaces coupent cet enchevêtrement : chaque module a **un point d'entrée** et un seul.

Exemple vu en cours (Java). Le module de commandes est un package `orders`. Une seule classe (le *repository* ou DAO) a le droit d'accéder à la table des commandes (insertion, mise à jour, suppression). Une classe *service* expose aux autres modules les opérations autorisées sans montrer l'interne. Quand on passe une commande, il faut diminuer la quantité en stock d'un produit : on passe par le service du module Produits, **jamais** directement par sa table.

```
// module produits : seul point d'entrée autorisé
public interface ProduitService {
    ProduitInfo trouver(long id);
    void reserverStock(long id, int quantite);
}
// module orders : utilise l'interface, pas la table « produits »
class CommandeService {
    private final ProduitService produits; // une interface
    private final CommandeRepository repo; // seul accès à sa table
    void passer(Commande c) {
        produits.reserverStock(c.produitId(), c.quantite());
        repo.inserer(c);
    }
}
```

Exemple vu en cours (Authentification). Un module Authentification isolé possède tout ce qui le concerne : comptes, connexion, changement de mot de passe. Aucun autre module ne touche ces données ; ils demandent à Authentification. Une équipe (par exemple un groupe de 15 développeurs) peut alors travailler sur ce module à sa façon, tant qu'elle respecte les points d'entrée qu'elle expose aux autres.

Ce qu'on garde du monolithe. Comme tout tourne dans un seul processus avec une seule base, les appels entre modules sont de simples appels de méthode (rapides, sans réseau) et une

opération qui touche plusieurs modules peut rester une **transaction atomique** : on garde une **forte cohérence** des données. En contrepartie, on garde aussi les limites du monolithe : on déploie et on met à l'échelle **tout ensemble**.

Ce qu'on prépare pour l'avenir. Des frontières nettes facilitent les évolutions module par module (par exemple la mise à jour de version du framework) et, si un jour c'est nécessaire, l'**extraction d'un module en microservice** : il suffira de remplacer l'appel d'interface par un appel réseau.

Pièges et cas particuliers

- **Des dossiers ne font pas des modules.** Ranger le code en packages `orders`, `products`… ne suffit pas si chaque package lit la table de l'autre : le couplage reste le même qu'un monolithe classique.
- **Modulaire ≠ déploiement séparé.** Les modules ne sont pas déployés indépendamment ; corriger un module oblige toujours à reconstruire et redéployer l'unique application.
- **Modulaire ≠ scalabilité indépendante.** On ne peut pas multiplier le seul module catalogue : on scale l'application entière (N3).
- **Une interface qui expose l'interne** (renvoyer l'entité JPA ou la connexion à la base) annule l'intérêt de la frontière.
- **Découper par couche technique** (contrôleurs / services / DAO) n'est pas découper par capacité métier : la règle demande des modules métier.

6. Le problème que la règle résout

Le problème

Un monolithe qui grandit devient un « plat de spaghettis » : chaque partie appelle directement les classes et les tables des autres. Une modification du catalogue casse la logique des commandes, plusieurs équipes se gênent dans le même code, et plus personne ne sait ce qu'il a le droit de toucher (signes « complexité » et « coordination des équipes » de N2). Pourtant, passer aux microservices serait disproportionné : il faudrait gérer un réseau, plusieurs déploiements, la cohérence entre bases — sans en avoir le besoin ni les compétences.

Le monolithe modulaire résout le **problème du couplage** sans créer la **complexité distribuée** : il met de l'ordre (frontières, propriétaires, interfaces) en gardant la simplicité d'un seul

déploiement et la cohérence d'une seule base.

7. À quoi elle sert

L'utilité pratique

- **Limiter le couplage accidentel** : une modification interne à un module ne casse pas les autres tant que son interface ne change pas.
- **Donner une propriété claire aux équipes** : chaque équipe possède un module et ses données, et ne touche les autres que par leurs interfaces.
- **Garder la simplicité d'exploitation** : un seul build, un seul déploiement, une seule base, des transactions atomiques.
- **Préparer une migration** : des frontières bien définies permettent plus tard d'extraire un module en microservice, sans tout réécrire.

8. D'où vient la règle

Le raisonnement qui y mène

Partons des deux notions précédentes. Le monolithe (N2) a des *avantages* qui viennent de son unité (développement, déploiement et tests simples, communication efficace) et des *limites* dont une partie vient de son **organisation interne** (complexité croissante, équipes qui se gênent) et une autre de son **déploiement unique** (redéploiement global, scalabilité en bloc).

Question : peut-on corriger les limites dues à l'organisation sans toucher au déploiement ? Oui, en découpant *à l'intérieur* :

- Découper par **capacité métier** ⇒ chaque module a une responsabilité claire ⇒ *organisation modulaire*.
- Interdire l'accès direct à l'interne des autres ⇒ on ne passe que par ce qu'ils exposent ⇒ *frontières claires via interfaces* ⇒ *couplage faible*.
- Garder un seul processus ⇒ les interfaces sont de simples appels de méthode ⇒ *communication simple*, et une seule base ⇒ *forte cohérence*.
- Garder une seule unité ⇒ *déploiement unique* ⇒ les limites liées au déploiement (scalabilité en bloc, redéploiement global) **restent**.

Les critères « quand le choisir » s'en déduisent : on le choisit quand on a besoin d'ordre mais **pas** de déploiements indépendants (taille moyenne, petites ou moyennes équipes, déploiement simple, cohérence forte), et quand les frontières métier ne sont pas encore assez stables pour les figer en services séparés.

9. Où l'utiliser

OÙ ? — Les contextes d'application

- **Applications métier de taille moyenne** (commerce en croissance, gestion d'école, ERP de PME) : plusieurs domaines fonctionnels distincts, mais une charge modérée et homogène.
- **Projets dont le métier évolue encore** (start-up qui a trouvé son marché mais ajuste son offre) : les frontières bougent, il est moins coûteux de les déplacer dans un même code qu'entre services en réseau.
- **Domaines à transactions fortes** (commande + réservation de stock, inscription + paiement) : la cohérence d'une seule base est précieuse.
- **Refonte d'un monolithe existant** : première étape pour démêler un code enchevêtré avant d'envisager, éventuellement, des microservices.
- **Analyse d'architecture** : au LAB 1, c'est l'architecture attendue pour le cas 2 (Growing Retailer).

10. Quand l'utiliser

QUAND ? — Les situations et leurs indices

Critère	Indices dans l'énoncé	Lecture
Application de taille moyenne	« plusieurs domaines distincts », « dizaines de milliers d'utilisateurs », « le code devient difficile à suivre »	Il faut une séparation fonctionnelle claire, sans complexité distribuée.
Exigences évolutives	« le métier change souvent », « on ne sait pas encore où couper », « les besoins se précisent »	Les frontières métier ne sont pas assez stables pour justifier des microservices.

Critère	Indices dans l'énoncé	Lecture
Forte cohérence des données	« commande et stock doivent être cohérents », « transaction », « jamais de double vente »	Interactions fréquentes et transactions atomiques : une seule base est un atout.
Équipes petites à moyennes	« 5 à 15 développeurs », « deux ou trois équipes », « besoin de propriété claire »	Les développeurs travaillent sur des modules séparés dans un code partagé.
Déploiement simple	« fenêtre de maintenance hebdomadaire », « peu d'expertise d'exploitation »	Une seule application déployable est plus facile à gérer et à maintenir.
Migration future possible	« peut-être des microservices plus tard », « préparer la croissance »	Des frontières bien définies faciliteront l'extraction de services.

Contre-indices (penser à une autre architecture) : « un module subit ×20 de trafic, les autres non », « chaque équipe veut livrer à son rythme », « technologies différentes par service » ⇒ microservices (N5) ; « 3 développeurs, un seul domaine simple » ⇒ monolithe classique (N2).

11. Comment l'utiliser

COMMENT ? — Concevoir (ou justifier) un monolithe modulaire en 5 étapes

1. **Diagnostiquer le problème principal** : trafic, déploiement ou couplage ? Relever les symptômes dans l'énoncé. *Pourquoi :* le monolithe modulaire répond au couplage, pas à une surcharge localisée ni à un besoin de livraisons indépendantes. *Résultat :* le problème dominant et deux symptômes.
2. **Découper en modules par capacité métier** et donner à chacun une responsabilité claire et ses données. *Pourquoi :* c'est la caractéristique « organisation modulaire ». *Résultat :* la liste des modules et de ce que chacun possède.
3. **Définir les interfaces internes** pour chaque dépendance (A → B : quelle opération B expose-t-il à A ?) et interdire l'accès direct aux tables des autres. *Pourquoi :* ce sont les « frontières claires » qui réduisent le couplage. *Résultat :* une table dépendance → opération autorisée.

4. **Vérifier les six critères « quand le choisir »** (taille, évolutivité, cohérence, équipes, déploiement, migration) et les contre-indices. *Pourquoi :* confirmer qu'un seul déploiement suffit. *Résultat :* les critères satisfaits / non satisfaits.
5. **Conclure et nommer la limite** : retenir le monolithe modulaire (ou une autre architecture si les contre-indices dominent), citer un inconvénient et un **déclencheur mesurable** de révision. *Pourquoi :* l'architecture reste déployée et mise à l'échelle en bloc. *Résultat :* une décision justifiée et sa condition de révision.

À retenir

Monolithe modulaire = modules métier faiblement couplés, qui ne se parlent que par des interfaces internes, dans **une seule** application déployée et mise à l'échelle en bloc. Il règle le couplage, garde la simplicité et la cohérence du monolithe, et prépare une éventuelle migration vers les microservices.

12. Exercices d'application corrigés

Exercice 4.1 — La plateforme scolaire

Une société édite une plateforme de gestion d'écoles privées (inscriptions, emplois du temps, notes, facturation). 8 développeurs en deux équipes, 20 000 utilisateurs, trafic régulier. Depuis peu, une modification des notes a cassé la facturation, car la facturation lit directement la table des notes pour calculer les remises au mérite. L'application est déployée chaque dimanche soir et l'équipe n'a aucune expérience du cloud. Quelle architecture recommander ?

Solution détaillée

Étape 1 — Diagnostic. Le trafic est régulier et le déploiement hebdomadaire convient : le problème est le **couplage**. Symptômes : « une modification des notes a cassé la facturation » et « la facturation lit directement la table des notes ».

Étape 2 — Modules. Inscriptions (élèves, classes), Emplois du temps (créneaux, salles), Notes (évaluations, moyennes), Facturation (frais, remises, paiements). Chaque module possède ses tables.

Étape 3 — Interfaces. Facturation → Notes : `NotesService.moyenneAnnuelle(eleveId)` au lieu de lire la table ; Emplois du temps → Inscriptions : `InscriptionService.elevesDeLaClasse(classeId)`. Si Notes change sa table, seule l'implémentation de l'interface est modifiée et la facturation n'est pas touchée.

Étape 4 — Critères. Taille moyenne (4 domaines), équipes petites (8 devs), déploiement simple (dimanche soir, pas d'expertise cloud), cohérence utile (remise calculée sur des notes à jour). Aucun contre-indice : pas de module surchargé, pas de besoin de livraisons séparées.

Étape 5 — Conclusion. Monolithe modulaire. Inconvénient : toute correction oblige à redéployer l'ensemble. Déclencheur : si une équipe doit livrer plus d'une fois par semaine sans attendre les autres, réévaluer.

Exercice 4.2 — Le cabinet médical

Un cabinet médical de 3 médecins fait développer par un seul développeur une application de prise de rendez-vous (agenda et fiches patients). 200 utilisateurs, un seul domaine simple, pas de projet de croissance. Le développeur propose « un monolithe modulaire pour faire propre ». Est-ce justifié ?

Solution détaillée

Étape 1 — Diagnostic. Pas de problème de trafic (200 utilisateurs), ni de déploiement, ni de couplage constaté : le code est petit et maintenu par une seule personne.

Étape 2 — Modules. On pourrait distinguer Rendez-vous et Patients, mais ces deux capacités sont très liées et réduites.

Étape 3 — Interfaces. Définir une interface entre deux petits modules gérés par la même personne apporte peu : il n'y a ni équipes à séparer ni enchevêtrement à prévenir.

Étape 4 — Critères. Ce n'est pas une application « de taille moyenne », il n'y a pas d'équipes à coordonner ni de migration prévue. Les critères correspondent plutôt au **monolithe classique** (petite application, petite équipe, N2).

Étape 5 — Conclusion. Un **monolithe** simple suffit ; un minimum d'organisation du code reste une bonne pratique, mais formaliser des frontières modulaires n'est pas nécessaire. Déclencheur : si de nouveaux domaines (facturation, téléconsultation) et d'autres développeurs arrivent, passer au monolithe modulaire.

Exercice 4.3 — Le grossiste et ses dépendances

Un grossiste a un monolithe en modules Catalogue, Commandes, Stock et Paiements. On constate que Commandes met à jour directement la table `stock`, et que Catalogue lit la table `stock` pour afficher la disponibilité. (a) Pour les dépendances Commandes → Stock, Commandes → Paiements et Catalogue → Stock, proposez une interface interne autorisée. (b) Énoncez deux règles d'accès aux données. (c) Une commande doit réserver le stock et enregistrer la commande « tout ou rien » : est-ce possible dans cette architecture ?

Solution détaillée

Étape 1 — Diagnostic. Problème de **couplage** : deux modules écrivent ou lisent directement les tables d'un autre module (Stock).

Étape 2 — Modules et données. Catalogue possède les fiches produits ; Commandes possède les commandes ; Stock possède les quantités ; Paiements possède les transactions.

Étape 3 — Interfaces (a). Commandes → Stock : `StockService.reserver(produitId, qte)` et `liberer(produitId, qte)` ; Commandes → Paiements : `PaiementService.payer(commandeId, montant)` ; Catalogue → Stock : `StockService.disponible(produitId)` (lecture seule). **Règles (b)** : (1) seul le *repository* d'un module accède à ses tables ; (2) un module n'appelle un autre module que par son interface de service, jamais ses classes internes.

Étape 4 — Critères (c). Oui : tous les modules tournent dans le même processus et partagent une seule base ; l'appel `reserver` puis l'insertion de la commande peuvent faire partie de la **même transaction atomique**. C'est la « forte cohérence » que permet le monolithe modulaire.

Étape 5 — Conclusion. Les interfaces suppriment le couplage par les tables tout en gardant la transaction ; l'inconvénient reste le déploiement et la mise à l'échelle en bloc.

13. Exercices d'entraînement

Exercice 4.4

Donnez la définition du monolithe modulaire, puis citez et expliquez en une phrase ses quatre caractéristiques clés.

Exercice 4.5

Citez les six situations dans lesquelles choisir un monolithe modulaire, avec pour chacune un indice qu'on pourrait lire dans un énoncé.

Exercice 4.6

Une agence de voyages en ligne (vols, hôtels, réservations, paiements ; 10 développeurs ; trafic modéré ; une modification des hôtels casse régulièrement les réservations) hésite sur son architecture. Appliquez la méthode en 5 étapes.

Exercice 4.7

Une bibliothèque universitaire (prêts, catalogue, adhérents, pénalités ; 4 développeurs ; un prêt et la mise à jour des pénalités doivent être cohérents) veut réorganiser son monolithe. Appliquez la méthode en 5 étapes.

14. Exercices de difficulté croissante

Facile Exercice 4.8

Vrai ou faux, en justifiant : « Dans un monolithe modulaire, on peut redéployer le seul module Paiements sans toucher aux autres. »

Facile Exercice 4.9

Pour chaque caractéristique, indiquez si elle est partagée par le monolithe (N2) et le monolithe modulaire, ou propre au monolithe modulaire : (a) unité de déploiement unique ; (b) organisation par capacité métier ; (c) communication dans le même processus ; (d) interfaces internes bien définies.

Moyen Exercice 4.10

Expliquez, à l'aide de l'analogie « Ahmed » vue en cours, ce qui se passe quand les modules s'importent directement les uns les autres dans une application qui grandit, et comment les interfaces internes règlent le problème.

Moyen Exercice 4.11

Un développeur a rangé son monolithe en packages `controllers`, `services` et `dao` et affirme avoir « un monolithe modulaire ». Discutez cette affirmation et proposez un découpage conforme à la règle pour une boutique en ligne.

Difficile Exercice 4.12

Reprenez l'enseigne de la section 2. Montrez quels problèmes le passage au monolithe modulaire réglerait (équipes qui se gênent, code enchevêtré) et lesquels il ne réglerait pas (Black Friday, service Python, redéploiement global), en justifiant chaque point par une caractéristique.

Difficile **Exercice 4.13**

Une entreprise utilise un monolithe modulaire depuis deux ans. Proposez trois déclencheurs **mesurables** qui justifieraient d'extraire un module en microservice, et expliquez pourquoi des frontières bien définies rendent cette extraction plus facile qu'à partir d'un monolithe classique.

Notion 5

L'architecture microservices

4. La règle

QUOI ? — Définition et principes fondamentaux

Définition. L'**architecture microservices** est un style d'architecture logicielle dans lequel une application est organisée comme une **collection de petits services faiblement couplés et déployables indépendamment**, chacun responsable d'une **capacité métier** précise.

Principe fondamental	Ce qu'il signifie
Responsabilité métier unique	Chaque service se concentre sur une capacité métier bien définie.
Couplage faible	Les services réduisent au minimum leurs dépendances les uns envers les autres.
Flexibilité technologique	Les services peuvent utiliser des technologies différentes quand c'est pertinent.
Déploiement indépendant	Un service peut être mis à jour et déployé **sans redéployer toute l'application**.
Scalabilité indépendante	Chaque service peut être mis à l'échelle selon sa propre demande.
Communication entre services	Les services interagissent par des interfaces bien définies, typiquement des **API** ou de la **messagerie**.

En pratique. Chaque service implémente une capacité métier précise et peut être **développé, déployé et mis à l'échelle indépendamment**.

Le prix à payer. Les microservices apportent plus d'indépendance de déploiement et de mise à l'échelle, mais introduisent la **complexité des systèmes distribués**.

Quand le choisir : applications grandes et complexes · autonomie des équipes · scalabilité indépendante · flexibilité technologique · infrastructure mature.

5. Explication de la règle

Ce que dit vraiment la règle

On garde les **mêmes modules** que dans le monolithe modulaire (Commandes, Produits, Paiements, Authentification…), mais on va jusqu'au bout de la séparation : chaque module devient un **programme à part**, développé, déployé et exécuté **indépendamment**, souvent sur sa propre machine. L'application n'est plus *une* unité de déploiement, mais **plusieurs**. Ce style vise les grandes organisations, avec des équipes de 50 ou 100 développeurs.

Le déploiement indépendant est la caractéristique clé. Exemple vu en cours : « vous avez un an pour livrer toute l'application, mais le catalogue doit être opérationnel la semaine prochaine ». En monolithe, c'est impossible : c'est tout ou rien, on ne déploie pas une partie de l'application. En microservices, on met en production le service Catalogue seul, puis les autres quand ils sont prêts.

La communication change de nature. Les modules ne s'appellent plus par des appels de méthode dans un même processus, mais par le **réseau** (API REST, messagerie). L'interface interne du monolithe modulaire devient une API :

```
// monolithe modulaire (N4) : appel de méthode dans le même processus
ProduitInfo p = produits.trouver(id);
// microservices : appel réseau vers le service Produits, déployé ailleurs
ProduitInfo p = http.get("http://produits/api/produits/" + id, ProduitInfo.class);
```

Conséquence : chaque service devient une porte ouverte sur le réseau. Il faut donc le **sécuriser**, **assainir les entrées** qu'il reçoit et prévenir les attaques comme l'**injection SQL**, ce qui n'était pas nécessaire pour un simple appel de méthode.

Scalabilité indépendante — l'exemple Massar. Le jour des résultats du baccalauréat, seule la consultation des résultats est prise d'assaut. En microservices, on ne fait grossir **que** le service des résultats (plus d'instances, N3), le reste de la plateforme garde ses ressources habituelles.

Flexibilité technologique. Les services sont agnostiques vis-à-vis de la technologie : une application Java peut s'appuyer sur un service de recommandation par IA écrit en **Python**, déployé à part ; les deux communiquent par le réseau. Le même raisonnement vaut pour le matériel : un module d'IA qui a besoin d'un **GPU** est hébergé seul sur un serveur GPU coûteux, tandis que les autres services tournent sur des serveurs ordinaires.

Chaque service possède ses données. Chaque service peut avoir sa propre base de données, à laquelle les autres n'accèdent pas directement. Exemple du paiement : le traitement de la carte

bancaire n'est pas fait par l'application elle-même, mais par le système bancaire, avec lequel le service Paiements communique.

L'API gateway, porte d'entrée. Analogie vue en cours : une université a une porte d'entrée contrôlée ; sans elle, on entrerait dans les salles directement depuis la rue. Sans passerelle, chaque service est exposé : un attaquant qui lit les requêtes dans l'onglet réseau de l'application mobile peut viser directement le service et la base des produits. L'**API gateway** est le point d'entrée unique qui contrôle les accès et les rôles avant de transmettre la requête au bon service.

Isolation des pannes. Les services tournent sur des serveurs différents (serveur A, serveur B…). Si l'on coupe l'électricité d'un serveur, l'application continue de tourner : seul le module hébergé sur ce serveur est arrêté. En monolithe, la même coupure arrête tout.

Aspect	Architecture monolithique	Architecture microservices
Structure	Toutes les fonctionnalités appartiennent à une seule application.	Les fonctionnalités sont organisées en services indépendants.
Déploiement	Toute l'application est déployée ensemble.	Chaque service est déployé indépendamment.
Communication	Appels directs de fonctions / méthodes internes.	Communication par API ou messagerie.
Scalabilité	Toute l'application est mise à l'échelle en bloc.	Chaque service est mis à l'échelle selon sa demande.
Technologie	En général une pile technologique partagée.	Les services peuvent utiliser des technologies différentes.
Gestion des données	Souvent une base de données partagée.	Chaque service possède ses données.
Isolation des pannes	Une panne peut affecter toute l'application.	Les pannes peuvent être isolées, mais peuvent se propager entre services.

Pièges et cas particuliers

- **« Micro » ne veut pas dire « petit en lignes de code ».** Un service se définit par **une capacité métier**, pas par sa taille ; découper une fonctionnalité en dix minuscules services qui s'appellent sans cesse recrée le couplage, en pire (par le réseau).
- **Isolées ≠ impossibles à propager.** Si Commandes attend Paiements et que Paiements ne répond plus, Commandes peut se bloquer à son tour : les pannes « peuvent se propager entre services ».
- **Une base partagée annule l'indépendance.** Plusieurs services qui lisent et écrivent les mêmes tables ne peuvent plus évoluer ni se déployer séparément : c'est un monolithe distribué.
- **Pas de transaction atomique gratuite.** Une opération qui touche deux services (commande + paiement) traverse le réseau et deux bases : la forte cohérence du monolithe n'est plus automatique.
- **Sans infrastructure mature, c'est un piège.** Déploiements multiples, surveillance, réseau, sécurité de chaque API : sans équipe et outils pour les gérer, les microservices coûtent plus qu'ils ne rapportent.

6. Le problème que la règle résout

Le problème

Reprenons l'enseigne de la section 2. Même bien rangée en monolithe modulaire (N4), elle reste **une seule unité** : au Black Friday, on ne peut pas renforcer le seul catalogue, il faut multiplier toute l'application ; le moteur de recommandation en Python ne peut pas vivre dans le même processus Java ; une petite correction des comptes oblige encore à redéployer l'ensemble ; et cinq équipes doivent toujours se synchroniser sur une même livraison.

Ces problèmes ne viennent plus de l'organisation du code, mais du **déploiement unique**. Les microservices les résolvent en séparant aussi l'exécution : chaque capacité métier est déployée, mise à l'échelle, écrite dans sa technologie et livrée à son rythme, indépendamment des autres.

7. À quoi elle sert

L'utilité pratique

- **Livrer par morceaux** : mettre en production un service (le catalogue la semaine prochaine) sans attendre ni redéployer le reste.
- **Mettre à l'échelle là où est la charge** : multiplier le seul service surchargé (résultats du bac, catalogue du Black Friday, service LLM) et économiser sur le reste.
- **Choisir la bonne technologie et le bon matériel par service** : Python pour l'IA, Java pour la gestion, GPU uniquement là où il sert.
- **Rendre les équipes autonomes** : chaque équipe possède un service, ses données et son rythme de livraison.
- **Contenir les pannes** : la chute d'un serveur n'arrête qu'un service, pas toute l'application.

8. D'où vient la règle

Le raisonnement qui y mène

Les trois scénarios de N1 posaient trois besoins : *servir plusieurs plateformes*, *intégrer des technologies différentes* et *mettre à l'échelle une seule partie*. Le monolithe (N2) n'y répond pas à cause de son unité ; le monolithe modulaire (N4) corrige l'organisation interne mais garde le déploiement unique. Il reste donc à séparer **le déploiement lui-même** :

- Un programme par capacité métier ⇒ *responsabilité métier unique*.
- Des programmes séparés ne partagent ni mémoire ni tables ⇒ ils ne se parlent que par des API ou de la messagerie ⇒ *communication entre services* et *couplage faible* ; chacun *possède ses données*.
- Des programmes séparés se déploient séparément ⇒ *déploiement indépendant* ; ils tournent sur des machines séparées ⇒ *scalabilité indépendante* (N3 appliquée service par service) et *isolation des pannes*.
- Comme ils ne communiquent que par le réseau, rien n'oblige à utiliser le même langage ⇒ *flexibilité technologique*.
- Mais le réseau, les déploiements multiples et les bases séparées ont un coût ⇒ *complexité des systèmes distribués* ⇒ il faut une *infrastructure mature*.

On obtient ainsi trois façons d'organiser et de déployer une application :

Approche	Description	En un mot
Monolithe	Toutes les fonctionnalités sont empaquetées et déployées ensemble, souvent avec des composants fortement intégrés.	Application unifiée
Monolithe modulaire	Les fonctionnalités sont organisées en modules bien définis, mais restent une seule application déployable.	Modulaire, déploiement unifié
Microservices	Les fonctionnalités sont séparées en services déployables indépendamment, qui communiquent par le réseau.	Services distribués

9. Où l'utiliser

OÙ ? — Les contextes d'application

- **Grandes plateformes grand public** (marketplace nationale, streaming, livraison) : nombreuses capacités métier, charges très inégales, plusieurs canaux (web, mobile, TV) servis par les mêmes API.
- **Plateformes publiques à pics ciblés** (résultats d'examens, déclarations fiscales) : une fonctionnalité subit un pic énorme à date connue, les autres non.
- **Systèmes intégrant de l'IA** : un service de recommandation, de détection de fraude ou un LLM écrit en Python et hébergé sur GPU, à côté d'un cœur métier en Java.
- **Organisations à nombreuses équipes** (50 à 100 développeurs et plus) qui doivent livrer chacune à son rythme.
- **Analyse d'architecture** : au LAB 1, c'est l'architecture attendue pour le cas 3 (National Marketplace).

10. Quand l'utiliser

QUAND ? — Les situations et leurs indices

Critère	Indices dans l'énoncé	Lecture
Applications grandes et complexes	« des dizaines de domaines », « millions d'utilisateurs », « plusieurs canaux »	Plusieurs capacités métier demandent une gestion indépendante.
Autonomie des équipes	« 100 développeurs », « une dizaine d'équipes », « chaque équipe veut livrer à son rythme »	Des équipes différentes développent, maintiennent et déploient leurs services indépendamment.
Scalabilité indépendante	« ×20 de trafic sur la recherche », « pic le jour des résultats », « le reste est stable »	Les services ont des charges et des besoins de mise à l'échelle différents.
Flexibilité technologique	« modèle en Python », « besoin d'un GPU », « une équipe en Dart, une autre en Java »	Des services différents gagnent à utiliser des technologies différentes.
Infrastructure mature	« équipe DevOps / plateforme », « conteneurs, CI/CD », « supervision 24 h/24 »	L'organisation sait gérer la complexité des systèmes distribués.

Règle de lecture : les cinq conditions sont à vérifier une par une ; l'**infrastructure mature** est une condition nécessaire — sans elle, on ne choisit pas les microservices, même si les autres indices sont présents. Ensuite, plus les critères satisfaits sont nombreux et forts, plus le choix est justifié.

Contre-indices (penser à une autre architecture) : « 3 à 5 développeurs », « pas d'équipe d'exploitation », « trafic modéré et homogène » ⇒ monolithe (N2) ; « besoin d'ordre et de cohérence forte, une seule fenêtre de déploiement » ⇒ monolithe modulaire (N4).

11. Comment l'utiliser

COMMENT ? — Justifier (ou écarter) les microservices en 5 étapes

1. **Diagnostiquer les besoins différenciés** : relever dans l'énoncé les capacités métier et ce qui les distingue (charge, technologie, matériel, rythme de livraison, équipe). *Pourquoi :* les

microservices ne se justifient que si les parties ont des besoins *différents*. *Résultat :* la liste des capacités et leurs différences.

2. **Vérifier les cinq critères « quand le choisir »**, en commençant par l'infrastructure mature, puis les contre-indices. *Pourquoi :* la complexité distribuée n'est acceptable que si l'organisation peut la gérer. *Résultat :* critères satisfaits / non satisfaits ; si l'infrastructure manque, on s'arrête et on conclut N2 ou N4.
3. **Découper en services par capacité métier**, chacun avec ses propres données. *Pourquoi :* principes « responsabilité unique » et « couplage faible ». *Résultat :* la liste des services, leur propriétaire et leur base.
4. **Définir la communication et la protection** : quelles API ou quels messages entre services, une API gateway en entrée, la validation des entrées de chaque API. *Pourquoi :* toute interaction passe désormais par le réseau. *Résultat :* un schéma gateway → services → bases.
5. **Conclure et nommer le coût** : relier chaque besoin du point 1 au principe qui le satisfait (déploiement, scalabilité, technologie, autonomie, pannes), puis citer un inconvénient (complexité distribuée, cohérence, sécurité) et une condition de révision. *Résultat :* une décision justifiée, avec ses limites.

À retenir

Microservices = une collection de petits services, **un par capacité métier**, faiblement couplés, chacun avec ses données, qui communiquent par API ou messagerie et sont **déployés et mis à l'échelle indépendamment**. On gagne l'indépendance (déploiement, charge, technologie, équipes, pannes) ; on paie la complexité des systèmes distribués, qui exige une infrastructure mature.

Question ouverte (prochaine séance)

Comment une application doit-elle être **découpée** en microservices indépendants, et **où** tracer les frontières entre eux ?

12. Exercices d'application corrigés

Exercice 5.1 — La plateforme de livraison de repas

Une plateforme nationale de livraison de repas gère le catalogue des restaurants, les commandes, les paiements, le suivi GPS des livreurs et un moteur de recommandation écrit en Python par l'équipe data. 120 développeurs répartis en 9 équipes. À midi et à 20 h, les commandes et le suivi des livreurs reçoivent huit fois plus de requêtes ; le catalogue reste stable. Une équipe plateforme gère déjà les conteneurs, les pipelines CI/CD et la supervision. Appliquez la méthode du 11 pour décider de l'architecture.

Solution détaillée

Étape 1 — Besoins différenciés. Cinq capacités : Catalogue, Commandes, Paiements, Suivi livreurs, Recommandation. Elles se distinguent par la **charge** (Commandes et Suivi ×8 aux heures de repas, Catalogue stable), par la **technologie** (Recommandation en Python, le reste non) et par les **équipes** (9 équipes qui ont chacune leur périmètre).

Étape 2 — Cinq critères. Infrastructure mature : *oui* (équipe plateforme, conteneurs, CI/CD, supervision) — la condition nécessaire est remplie. Grande application : *oui* (cinq domaines, plateforme nationale). Autonomie des équipes : *oui* (120 développeurs, 9 équipes). Scalabilité indépendante : *oui* (pics localisés). Flexibilité techno : *oui* (Python). Aucun contre-indice (pas de petite équipe, charge non homogène).

Étape 3 — Services. Un service par capacité : `catalogue`, `commandes`, `paiements`, `suivi-livreurs`, `recommandation` (Python). Chacun possède sa base : les commandes ne lisent pas la table du catalogue, elles appellent son API.

Étape 4 — Communication. Les applications mobiles et le site passent par une **API gateway** qui contrôle l'authentification et les rôles (un livreur n'accède pas aux paiements). Derrière, les services communiquent par API REST ou messages ; chaque API valide ses entrées.

Étape 5 — Conclusion. Architecture **microservices** : on multiplie seulement Commandes et Suivi aux heures de repas (scalabilité indépendante), la recommandation vit en Python (flexibilité), chaque équipe livre son service sans redéployer les autres (déploiement indépendant, autonomie), et la panne du suivi n'empêche pas de commander (isolation). *Inconvénient :* une commande payée touche deux services et deux bases, la cohérence n'est plus garantie par une transaction unique.

Exercice 5.2 — Le jour des résultats

Un ministère exploite une plateforme nationale : inscriptions aux examens, saisie des notes par les établissements, consultation des résultats par les élèves, statistiques. Le jour de la publication des résultats, la consultation reçoit cinquante fois plus de requêtes que d'habitude, les autres fonctions restent normales. Aujourd'hui tout est dans un monolithe et, ce jour-là, la saisie des notes devient aussi inaccessible. Le ministère dispose d'une équipe d'exploitation expérimentée. Expliquez, avec la méthode du 11, ce qu'apporteraient les microservices.

Solution détaillée

Étape 1. Quatre capacités : Inscriptions, Saisie des notes, Consultation des résultats, Statistiques. La différence décisive est la **charge** : un pic ×50 sur la seule consultation, à une date connue. Le symptôme actuel (la saisie tombe aussi) montre que, dans le monolithe, une surcharge localisée se propage à toute l'application.

Étape 2. Infrastructure mature : *oui* (équipe d'exploitation expérimentée). Grande application nationale : *oui*. Scalabilité indépendante : *oui*, c'est le besoin principal. Autonomie des équipes et flexibilité techno : non mentionnées, mais leur absence n'empêche pas la décision tant que l'infrastructure est mature et que le besoin de charge est fort.

Étape 3. Services `inscriptions`, `notes`, `resultats`, `statistiques`, chacun avec ses données ; le service des résultats peut recevoir une copie des notes publiées plutôt que lire la base de saisie.

Étape 4. Une API gateway reçoit le trafic des élèves et l'oriente vers `resultats` ; la saisie des notes, réservée aux établissements, est contrôlée par rôle à la passerelle et n'est pas exposée au public.

Étape 5. Les microservices permettent de **ne faire grossir que le service des résultats** le jour J (scaling out de ce seul service, N3), sans payer pour les autres ; et si ce service sature malgré tout, la saisie des notes, déployée ailleurs, continue de fonctionner (isolation des pannes). *Inconvénient :* il faut synchroniser les notes publiées entre deux services ; *révision :* si le pic disparaît, le gain ne justifierait plus cette complexité.

Exercice 5.3 — La salle de sport « moderne »

Une start-up de 4 développeurs crée une application de réservation de cours pour une chaîne de trois salles de sport (comptes, planning, réservations, abonnements). 2 000 adhérents, trafic régulier, pas d'équipe d'exploitation : les développeurs déploient eux-mêmes sur un serveur loué. Le responsable veut des microservices « parce que c'est moderne ». Appliquez la méthode du 11 et concluez.

Solution détaillée

Étape 1. Quatre capacités (Comptes, Planning, Réservations, Abonnements), mais **aucun besoin différencié** : même charge régulière, même technologie, une seule petite équipe, un seul rythme de livraison.

Étape 2. Infrastructure mature : *non* (pas d'équipe d'exploitation, déploiement manuel sur un serveur). La condition nécessaire n'est pas remplie : on s'arrête là. Les autres critères sont aussi absents : petite application, 4 développeurs, charge homogène, aucune technologie particulière. Les contre-indices « 3 à 5 développeurs » et « pas d'équipe d'exploitation » sont présents.

Étapes 3–4. Sans objet : on ne découpe pas en services. On peut tout de même noter que les quatre capacités sont des modules naturels à l'intérieur d'une application.

Étape 5. Les microservices ne sont **pas** justifiés : ils ajouteraient réseau, sécurité de quatre API, quatre déploiements et la cohérence entre bases (une réservation doit vérifier l'abonnement), sans aucun gain d'indépendance utile. On retient un **monolithe** (N2), éventuellement organisé en modules (N4) pour préparer la croissance. *Révision :* si la chaîne passe à des centaines de salles avec plusieurs équipes et une infrastructure dédiée, on réexamine les cinq critères. « C'est moderne » n'est pas un critère.

13. Exercices d'entraînement

Exercice 5.4

Donnez la définition de l'architecture microservices, puis citez et expliquez en une phrase chacun de ses six principes fondamentaux.

Exercice 5.5

Reconstituez le tableau de comparaison monolithe / microservices sur ses sept aspects (structure, déploiement, communication, scalabilité, technologie, données, isolation des pannes), en ajoutant pour chaque ligne un exemple tiré du cours.

Exercice 5.6

Une banque en ligne ajoute à son application Java un module de détection de fraude par IA, écrit en Python et qui nécessite un GPU. Les autres fonctions (comptes, virements, relevés) tournent sur des serveurs ordinaires. Indiquez quels principes des microservices sont exploités et expliquez l'avantage économique par rapport au monolithe.

Exercice 5.7

Expliquez le rôle d'une API gateway à l'aide de l'analogie de la porte d'entrée de l'université, et décrivez le risque concret lorsqu'une application mobile appelle directement chaque service.

14. Exercices de difficulté croissante

Facile Exercice 5.8

Vrai ou faux, en justifiant : « En microservices, la panne d'un service n'affecte jamais les autres services. »

Facile Exercice 5.9

Associez chaque expression à l'une des trois approches (monolithe, monolithe modulaire, microservices) : « application unifiée » ; « services distribués » ; « modulaire, déploiement unifié » ; « communiquent par le réseau » ; « composants fortement intégrés » ; « modules bien définis, une seule application déployable ».

Moyen Exercice 5.10

Une direction annonce : « Vous avez un an pour livrer toute l'application, mais le catalogue doit être opérationnel la semaine prochaine. » Expliquez pourquoi c'est impossible avec un monolithe ou un monolithe modulaire, et quel principe des microservices le rend possible.

Moyen Exercice 5.11

Pourquoi, en microservices, chaque service possède-t-il ses propres données ? Montrez la conséquence de ce choix sur l'opération « passer une commande et réserver le stock », comparée au monolithe modulaire (N4).

Difficile Exercice 5.12

Reprenez l'enseigne de la section 2. Montrez, demande par demande (plusieurs plateformes, moteur Python, Black Friday, correction des comptes, cinq équipes), quel principe des microservices y répond. Indiquez ensuite la condition que l'enseigne doit remplir avant de migrer et deux nouveaux problèmes que la migration créera.

Difficile Exercice 5.13

Question ouverte du cours : proposez un premier découpage de l'enseigne de la section 2 en services. Pour chaque service, donnez sa capacité métier et les données qu'il possède, puis expliquez deux critères que vous avez utilisés pour tracer les frontières et un risque d'un découpage trop fin.

Section 15

Problèmes réels et solutions détaillées

Comment lire cette section

Les quatre cas qui suivent sont ceux du **LAB 1 — Foundations**. Ils mettent en concurrence les trois architectures du cours : monolithe (N2), monolithe modulaire (N4) et microservices (N5). Pour chaque cas, on répond d'abord à toutes les questions, *puis* seulement on choisit. Le LAB le rappelle : les microservices ne sont pas supérieurs par nature ; un autre choix est acceptable si le raisonnement couvre toutes les contraintes.

Chaque choix final est défendu avec la **règle de défense** du LAB :

- **(a)** l'architecture choisie ;
- **(b)** trois faits *tirés du cas* qui la justifient ;
- **(c)** pourquoi les deux autres architectures conviennent moins ;
- **(d)** un inconvénient du choix ;
- **(e)** un déclencheur qui obligerait à revoir le choix.

Cas 1 — La librairie de quartier

Énoncé

Une librairie locale lance un site web pour **200 à 400 clients**. **Trois développeurs** construisent et maintiennent Authentification, Catalogue, Commandes, Paiements et Stock. Le catalogue change chaque semaine ; les achats sont peu fréquents. L'entreprise utilise **un serveur d'application et une base relationnelle**. Les commandes doivent mettre à jour le stock **immédiatement et de façon fiable**. L'équipe livre une version toutes les deux semaines, n'a **aucun personnel d'exploitation** et doit lancer le site **en six semaines**.

1. Quelles sont les deux exigences métier les plus critiques au lancement ? Citez le scénario.
2. Des modules ont-ils besoin d'une scalabilité ou d'un déploiement indépendants ? Sur quoi vous appuyez-vous ?
3. À quel point Commandes, Paiements et Stock doivent-ils se coordonner ? Que gagne-t-on à garder ces opérations dans une seule application ?
4. Quels coûts ou modes de panne supplémentaires les microservices apporteraient-ils à cette équipe ?
5. Une organisation interne du code plus claire serait-elle utile même avec un déploiement unique ? Expliquez.
6. Choisissez une architecture et défendez-la face aux deux autres : trois faits, un inconvénient, une condition qui changerait votre choix.

Solution détaillée — Cas 1

Q1 — Exigences critiques. La première est le **délai** : le site « doit être lancé en six semaines » avec trois développeurs seulement ; toute architecture qui alourdit le développement menace directement le lancement. La seconde est la **fiabilité de la commande** : « les commandes doivent mettre à jour le stock immédiatement et de façon fiable ». Vendre un livre qui n'est plus en stock, ou débiter un client sans enregistrer sa commande, serait l'erreur la plus grave pour une petite librairie.

Q2 — Scalabilité ou déploiement indépendants ? Non. Côté charge : 200 à 400 clients et des achats « peu fréquents » ; un seul serveur suffit et aucun module n'a de pic qui lui soit propre (c'est l'inverse du Black Friday de N1 et N3). Côté déploiement : une seule équipe de trois personnes livre *tous* les modules au même rythme (toutes les deux semaines) ; le catalogue

change chaque semaine, mais ce sont des *données* (nouveaux livres), pas du code, et elles se modifient sans redéployer.

Q3 — Coordination Commandes / Paiements / Stock. Elle est **forte** : une commande n'est valide que si le paiement est accepté *et* le stock décrémenté, les trois en même temps. Dans une seule application reliée à une seule base, ces trois écritures tiennent dans **une transaction atomique** : soit tout réussit, soit tout est annulé. C'est exactement le cas « transactions atomiques fréquentes » où le cours recommande le monolithe (N2) ; on gagne aussi des appels directs en mémoire, rapides et sans panne réseau.

Q4 — Coûts des microservices pour cette équipe. Cinq services signifieraient cinq déploiements, cinq bases, une communication réseau et une supervision à mettre en place — sans personnel d'exploitation et en six semaines. Nouveaux modes de panne : le service Stock ne répond pas pendant une commande (délai d'attente dépassé), le paiement est accepté mais la mise à jour du stock échoue (incohérence entre deux bases), une version d'un service casse l'API attendue par un autre. Le cours pose une *infrastructure mature* comme condition nécessaire des microservices ; elle est absente ici.

Q5 — Organisation interne claire ? Oui. Même avec un seul déploiement, séparer le code en modules (un paquet par domaine, une interface par module) rend le code plus lisible, limite les effets de bord entre modules et prépare une évolution future vers le monolithe modulaire si la librairie grandit. Cela ne coûte presque rien et ne change pas le déploiement.

Défense (a)–(e) — Cas 1

(a) Architecture : monolithe (une seule unité déployée, avec un code proprement rangé par domaine).

(b) Trois faits : ① trois développeurs, aucune équipe d'exploitation ; ② 200–400 clients et achats peu fréquents, sur un seul serveur ; ③ commande et stock doivent être mis à jour « immédiatement et de façon fiable », donc dans une même transaction.

(c) Pourquoi pas les autres : les *microservices* imposent une infrastructure et une expertise absentes, et cassent la transaction commande–stock, pour aucun besoin de scalabilité ou de déploiement indépendants. Le *monolithe modulaire* n'est pas faux, mais ses frontières strictes (accès aux données uniquement par interface) sont un effort que trois développeurs pressés par un délai de six semaines ne rentabiliseront pas encore : aucun problème de couplage entre équipes n'existe, puisqu'il n'y a qu'une équipe.

(d) Inconvénient : si le code grossit sans discipline, la complexité croîtra et toute modification obligera à redéployer l'ensemble ; une panne d'un module peut arrêter tout le site.

(e) Déclencheur : la croissance de l'équipe (plusieurs équipes qui se gênent) ou des modifications d'un module qui cassent régulièrement un autre → passer au monolithe modulaire (c'est exactement le Cas 2).

Cas 2 — Le commerçant en croissance

Énoncé

Un commerçant sert désormais **30 000 clients** avec une équipe de **12 développeurs**. Authentification, Catalogue, Commandes, Paiements et Stock partagent un seul code source. Des modifications fréquentes du Catalogue ont commencé à **casser la logique des Commandes**, parce que les modules **accèdent directement aux tables des autres**. Les équipes ont besoin d'une responsabilité plus claire et d'interfaces internes. Le trafic est **modéré et semblable** d'un module à l'autre. Tous les composants peuvent encore être déployés **ensemble pendant une fenêtre de maintenance hebdomadaire**. L'entreprise a une **expertise d'exploitation limitée** ; la prise de commande et la réservation du stock exigent une **cohérence forte**.

1. Identifiez le problème architectural principal : trafic, déploiement ou couplage. Citez deux symptômes.
2. Définissez cinq modules fonctionnels et donnez à chacun une responsabilité claire.
3. Pour chaque dépendance (Commandes→Stock, Commandes→Paiements, Catalogue→Stock), proposez une interface interne autorisée à la place de l'accès direct aux tables.
4. Le cas justifie-t-il des déploiements réseau indépendants ? Comparez bénéfices probables et complexité ajoutée.
5. Quelles frontières et règles d'accès aux données réduiraient le couplage accidentel tout en gardant une seule unité de déploiement ?
6. Dessinez l'architecture proposée : modules, interfaces internes, frontière de déploiement.
7. Choisissez une architecture, défendez-la face aux deux autres, donnez un inconvénient et un déclencheur de migration mesurable.

Solution détaillée — Cas 2

Q1 — Problème principal : le couplage. Ce n'est pas le trafic (« modéré et semblable d'un module à l'autre ») ni le déploiement (« tout peut encore être déployé ensemble » chaque semaine). Symptômes : ① « les modifications du Catalogue cassent la logique des Commandes » ; ② « les modules accèdent directement aux tables des autres ». S'y ajoute le besoin exprimé de « responsabilité plus claire et d'interfaces internes » : douze développeurs se marchent sur les pieds dans un code sans frontières.

Q2 — Cinq modules.

Module	Responsabilité unique	Données possédées
Authentification	identifier les utilisateurs et gérer leurs rôles	comptes, rôles
Catalogue	décrire les produits (fiches, prix, catégories)	produits, prix
Commandes	créer et suivre les commandes	commandes, lignes
Paiements	encaisser et rembourser	transactions
Stock	connaître et réserver les quantités disponibles	quantités, réservations

Q3 — Interfaces internes (appels de méthode dans le même processus, comme en N4) :

- **Commandes → Stock :** `reserver(produitId, quantite) → reservationId` et `annulerReservation(reservationId)`. Commandes ne lit plus la table des quantités.
- **Commandes → Paiements :** `payer(commandeId, montant) → resultat` et `rembourser(commandeId)`. Commandes ne touche pas la table des transactions.
- **Catalogue → Stock :** `disponibilite(produitId) → quantite` (lecture seule) pour afficher « en stock / épuisé » sur la fiche produit.

Ainsi, si le Catalogue change la structure de ses tables, rien ne casse ailleurs tant que son interface reste la même.

Q4 — Déploiements réseau indépendants ? Non. *Bénéfices probables* : faibles, car le trafic est homogène (pas de scalabilité indépendante à gagner) et la livraison hebdomadaire commune convient. *Complexité ajoutée* : forte — appels réseau qui peuvent échouer, une base par service, supervision distribuée, et surtout la perte de la transaction unique alors que commande et réservation du stock exigent une cohérence forte. Avec une expertise d'exploitation limitée, le coût dépasse largement le gain. Le vrai problème (couplage) se règle par des frontières, pas par le réseau.

Q5 — Règles de frontière. ① Chaque table appartient à un seul module ; un module n'accède jamais directement aux tables d'un autre. ② Un module n'expose qu'une interface publique ; ses classes internes sont privées au paquet. ③ Les dépendances suivent un sens défini (Commandes dépend de Stock et Paiements, jamais l'inverse) et les cycles sont interdits. ④ Une équipe propriétaire par module, qui valide toute modification de son interface. ⑤ La base reste unique (cohérence forte par transaction), mais organisée en schémas séparés par module.

Q6 — Schéma de l'architecture proposée.

```
=== UNE SEULE UNITÉ DE DÉPLOIEMENT (un processus) ==================

  [Authentification]      [Catalogue] --disponibilite()--> [Stock]

  [Commandes] --reserver() / annulerReservation()--------> [Stock]
  [Commandes] --payer() / rembourser()------------------> [Paiements]

  (appels de méthode in-process, via interfaces publiques uniquement)
=====================================================================
                              |
                              v
  Base relationnelle unique, un schéma par module :
  auth | catalogue | commandes | paiements | stock
  (chaque module n'accède qu'à son propre schéma)
```

Défense (a)–(e) — Cas 2

(a) Architecture : monolithe modulaire.

(b) Trois faits : ① le problème est le couplage (accès direct aux tables, Catalogue qui casse Commandes) ; ② le trafic est modéré et homogène et un déploiement commun hebdomadaire suffit ; ③ commande et réservation du stock exigent une cohérence forte, avec une expertise d'exploitation limitée.

(c) Pourquoi pas les autres : rester en *monolithe* classique laisse intact le couplage qui cause déjà des pannes, et douze développeurs ne peuvent pas se répartir les responsabilités sans frontières. Les *microservices* résoudraient un problème de scalabilité et de déploiement que le cas n'a pas, au prix d'une complexité d'exploitation que l'entreprise ne maîtrise pas et de la perte de la cohérence transactionnelle.

(d) Inconvénient : le déploiement reste unique : une correction urgente du Catalogue attend la fenêtre de maintenance, et une panne grave d'un module peut encore arrêter toute l'application.

(e) Déclencheur mesurable : par exemple, si un module concentre durablement plus de cinq fois le trafic des autres pendant les pics, ou si une équipe doit livrer plus d'une fois par semaine alors que la fenêtre commune est hebdomadaire, on extrait ce module en service (à condition d'avoir acquis l'expertise d'exploitation).

Cas 3 — La place de marché nationale

Énoncé

Une place de marché opère dans plusieurs régions et compte **150 ingénieurs** organisés en équipes indépendantes Catalogue, Commandes, Paiements, Identité et Stock. Le trafic du Catalogue est **multiplié par 20** pendant les soldes, celui des Paiements seulement **par 2**. Le Catalogue est mis à jour **chaque jour**, les Paiements changent **chaque mois** sous vérification stricte, et l'Identité a son **propre calendrier de mises à jour de sécurité**. Une panne du Catalogue ne doit pas empêcher le traitement des commandes existantes, dans la mesure du possible. L'organisation dispose d'une **supervision mature, d'un déploiement automatisé et d'ingénieurs expérimentés** en systèmes distribués. Les équipes ont besoin d'une **propriété séparée des données** et de calendriers de déploiement séparés.

1. Quelles contraintes indiquent le besoin de capacités métier déployables indépendamment ? Au moins trois.
2. Quels composants ont des besoins de scalabilité nettement différents ? Que gaspillerait la mise à l'échelle de toute l'application ?
3. Proposez des frontières de services pour Catalogue, Commandes, Paiements, Identité et Stock ; expliquez une frontière qui pourrait raisonnablement rester fusionnée.
4. Dessinez les interactions API entre services pour passer une commande. Indiquez quel service possède chaque jeu de données.
5. Identifiez deux nouveaux risques des systèmes distribués et une parade pour chacun.
6. Pourquoi un monolithe ou un monolithe modulaire conviendrait-il moins à cette organisation ?
7. Choisissez une architecture, défendez-la, reconnaissez un inconvénient et donnez une condition pour regrouper des services.

Solution détaillée — Cas 3

Q1 — Contraintes de déploiement indépendant. ① Rythmes de livraison différents : Catalogue chaque jour, Paiements chaque mois sous vérification stricte, Identité selon son calendrier de sécurité ; dans une unité unique, chaque mise à jour quotidienne du Catalogue obligerait à redéployer et revérifier les Paiements. ② Des équipes indépendantes qui demandent explicitement des calendriers de déploiement et une propriété des données séparés (autonomie des équipes). ③ L'exigence d'isolation des pannes : une panne du Catalogue ne doit pas bloquer

les commandes existantes, ce qu'un processus unique ne peut garantir. ④ Les charges très différentes (Q2).

Q2 — Scalabilité différenciée. Le Catalogue (×20 en soldes) et les Paiements (×2) ont des besoins très éloignés. Mettre à l'échelle toute l'application pour suivre le Catalogue reviendrait à lancer vingt fois plus d'instances de Paiements, Identité, Stock et Commandes qui n'en ont pas besoin : gaspillage de ressources, coûts inutiles, exactement le défi de la « scalabilité indépendante » vu en N1. En scalabilité horizontale par service (N3), on multiplie seulement les instances du Catalogue.

Q3 — Frontières de services. Un service par capacité métier, chacun avec sa base : `catalogue` (fiches, prix), `commandes` (commandes, lignes, statuts), `paiements` (transactions), `identite` (comptes, droits, jetons), `stock` (quantités, réservations). *Frontière qui pourrait rester fusionnée :* Commandes et Stock. Leurs données sont étroitement liées (une commande réserve du stock), leurs charges évoluent ensemble et la réservation exige une forte cohérence ; les garder dans un même service éviterait une cohérence entre deux bases. On ne les sépare que si les deux équipes ont réellement besoin de rythmes distincts — ce qui est le cas ici puisqu'il existe une équipe Stock.

Q4 — Passer une commande.

```
Client --> [API gateway] --jeton--> [Identité]   (comptes, droits)
               |
               v
           [Commandes]                          (commandes, statuts)
             1. GET  /produits/{id}  --> [Catalogue] (fiches, prix)
             2. POST /reservations   --> [Stock]     (quantités, réserv.)
             3. POST /paiements      --> [Paiements] (transactions)
             4. statut « confirmée »
                (ou annulation de la réservation si 3 échoue)

Entre parenthèses : données possédées. Chaque service n'écrit que
dans SA base.
```

Si le Catalogue est en panne, les commandes déjà créées continuent d'être payées et expédiées : elles n'ont plus besoin de lui (le prix a été recopié dans la commande au moment de l'achat).

Q5 — Deux risques distribués. ① *Panne partielle / lenteur* : le Stock ne répond plus et Commandes attend indéfiniment, ses requêtes s'accumulent et elle tombe à son tour. Parade : délais d'attente (timeouts) courts et réponse de repli (« commande en attente ») au lieu d'attendre. ② *Cohérence entre services* : le stock est réservé mais le paiement échoue, ou l'inverse ; aucune transaction unique ne couvre deux bases. Parade : une étape de compensation

explicite (annuler la réservation si le paiement échoue) et des statuts de commande intermédiaires.

Q6 — Pourquoi pas les deux autres. Un *monolithe* ou un *monolithe modulaire* reste une seule unité de déploiement : la livraison quotidienne du Catalogue redéploierait aussi les Paiements soumis à vérification stricte ; on ne pourrait multiplier que l'application entière (×20 pour tout) ; une panne du processus emporterait aussi les commandes existantes ; et cinq équipes indépendantes, 150 ingénieurs, se coordonneraient sur une seule livraison. Les frontières internes du monolithe modulaire règlent le couplage du code, pas ces contraintes-là.

Défense (a)–(e) — Cas 3

(a) Architecture : microservices.

(b) Trois faits : ① Catalogue ×20 contre Paiements ×2 en soldes ; ② calendriers de livraison quotidien, mensuel et sécurité, portés par des équipes indépendantes qui veulent leurs propres données ; ③ supervision mature, déploiement automatisé et expérience des systèmes distribués — la condition nécessaire du cours est remplie.

(c) Pourquoi pas les autres : le *monolithe* cumule tous les signes d'essoufflement (redéploiement global, scalabilité globale, coordination des équipes) ; le *monolithe modulaire* clarifie le code mais garde un seul déploiement, une seule mise à l'échelle et une seule zone de panne.

(d) Inconvénient : la cohérence entre Commandes, Stock et Paiements n'est plus garantie par une transaction ; il faut gérer les pannes partielles et les compensations, ce qui ajoute du code et de la supervision.

(e) Condition de regroupement : si deux services sont presque toujours modifiés et déployés ensemble (par exemple Commandes et Stock) ou si leurs appels mutuels deviennent le principal facteur de lenteur et d'incohérence, on les fusionne en un seul service.

Cas 4 — L'entreprise régionale (à défendre)

Énoncé

Une entreprise régionale de commerce en ligne fonctionne depuis **cinq ans** et sert environ **100 000 clients**. Son application — Authentification, Catalogue, Commandes, Paiements, Stock — est entièrement écrite en **Java** et déployée comme **une seule application**.

Elle emploie **25 développeurs en trois équipes**. Chaque équipe a ses fonctionnalités, mais les développeurs **modifient souvent le code des autres équipes** ; des changements du Catalogue ont parfois provoqué des **pannes inattendues dans les Commandes**.

Pendant les campagnes promotionnelles, le trafic du Catalogue est multiplié **jusqu'à 10**, tandis que Commandes et Paiements augmentent modérément. Ces campagnes n'ont lieu que **quatre fois par an**, et l'infrastructure actuelle absorbe le trafic normal sans difficulté.

L'entreprise livre **toutes les deux semaines**. La direction voudrait livrer le Catalogue plus souvent, mais Paiements et Stock demandent une **coordination soigneuse** pour garder la cohérence des données. Elle prévoit d'introduire l'an prochain un **système de recommandation IA en Python**.

L'équipe technique maîtrise la maintenance d'applications Java mais a une **expérience limitée des systèmes distribués**. La direction a approuvé un **budget d'infrastructure modéré** et attend une architecture qui accompagne la croissance des **trois prochaines années**.

Question : quelle architecture recommandez-vous — monolithe, monolithe modulaire ou microservices ?

Solution détaillée — Cas 4

Étape 1 — Classer les faits par architecture. Ce cas est volontairement partagé : certains faits poussent vers les microservices, d'autres les déconseillent.

Fait du cas	Ce qu'il indique
Développeurs qui modifient le code des autres ; Catalogue qui casse Commandes	problème de **couplage** → frontières de modules (N4)
Catalogue ×10 en campagne, Commandes/Paiements modérés	argument microservices… mais seulement **4 fois par an**, et le trafic normal est absorbé

Livrer le Catalogue plus souvent	argument microservices (déploiement indépendant)
Paiements et Stock exigent une coordination pour la cohérence	argument **contre** leur séparation : transaction unique souhaitable
Recommandation IA en Python l'an prochain	hétérogénéité technologique → un service séparé (N1, scénario 2)
Expérience limitée des systèmes distribués, budget modéré	la condition nécessaire des microservices (infrastructure mature) **n'est pas remplie**
25 développeurs, 3 équipes, horizon de 3 ans	taille moyenne : besoin de structure et d'un chemin de migration

Étape 2 — Éliminer le monolithe actuel. Il montre déjà deux des signes d'essoufflement du cours (N2) : la complexité (pannes inattendues causées par des modifications croisées) et la coordination difficile entre équipes. Le garder tel quel, c'est laisser ces pannes continuer pendant trois ans de croissance.

Étape 3 — Tester les microservices. Ils répondraient au pic du Catalogue, à la livraison plus fréquente et au module Python. Mais le critère nécessaire du cours — une infrastructure et une équipe capables d'exploiter un système distribué — n'est pas rempli, et le budget est modéré. Le pic ×10 n'arrive que quatre fois par an : il peut être absorbé en ajoutant temporairement des instances de l'application entière (scalabilité horizontale, N3) pendant les campagnes, ce qui coûte bien moins qu'une plateforme de microservices exploitée toute l'année. Enfin, séparer Paiements et Stock casserait la cohérence que l'énoncé demande de préserver.

Étape 4 — Retenir le monolithe modulaire. Il traite le vrai problème actuel (couplage entre équipes) avec des compétences que l'équipe possède déjà (Java, une seule unité) : trois équipes, des modules à frontières claires, accès aux données uniquement par interfaces, Paiements et Stock dans la même transaction. Le cours le présente précisément pour les applications moyennes, aux exigences évolutives, avec une forte cohérence et en vue d'une *migration future*.

Étape 5 — Traiter le module Python. Le système de recommandation ne peut pas vivre dans le processus Java. Comme dans le scénario 2 de N1, on le déploie comme un **service séparé appelé par HTTP** par le module Catalogue. Ce seul service distribué, non critique (si la recommandation tombe, on affiche simplement le catalogue sans suggestions), permet à l'équipe d'apprendre les systèmes distribués sans risquer les paiements.

Défense (a)–(e) — Cas 4

(a) Architecture : monolithe modulaire Java, avec la recommandation Python en service séparé appelé par HTTP.

(b) Trois faits : ① les développeurs modifient le code des autres équipes et le Catalogue casse les Commandes (couplage) ; ② l'équipe a une expérience limitée des systèmes distribués et un budget modéré ; ③ Paiements et Stock exigent une coordination pour la cohérence des données.

(c) Pourquoi pas les autres : le *monolithe* actuel ne règle pas le couplage qui provoque déjà des pannes et ne prépare pas trois ans de croissance. Les *microservices* exigent une maturité d'exploitation absente, coûtent plus que le budget modéré, cassent la cohérence Paiements–Stock, et le pic ×10 ne survient que quatre fois par an.

(d) Inconvénient : le Catalogue ne pourra pas être livré indépendamment : la demande de la direction (livrer le Catalogue plus souvent) n'est satisfaite que partiellement, et pendant les campagnes on multiplie toute l'application, pas seulement le Catalogue.

(e) Déclencheur : si les campagnes deviennent fréquentes (par exemple mensuelles) ou si le Catalogue doit être livré plusieurs fois par semaine, *et* que l'équipe a acquis l'expérience des systèmes distribués (notamment grâce au service de recommandation), on extrait le module Catalogue en premier microservice — ses frontières propres rendent cette extraction simple.

Piège — Cas 4

Voir « Catalogue ×10 » et « Python » et conclure aussitôt « microservices ». Le LAB demande de peser *toutes* les contraintes : un pic quatre fois par an, une équipe sans expérience distribuée et un budget modéré pèsent plus lourd. Une défense des microservices reste acceptable si elle explique comment l'entreprise acquerra cette maturité et comment elle préservera la cohérence Paiements–Stock.

Section 16

Banque d'exercices

Comment utiliser cette banque

Cette banque mélange les cinq notions du cours : motivations (N1), monolithe (N2), scalabilité (N3), monolithe modulaire (N4) et microservices (N5). Les exercices sont rangés par difficulté : dix faciles (définitions, vocabulaire, reconnaissance), dix moyens (application à une situation) et dix difficiles (choix argumenté d'une architecture, inspiré du LAB 1).

Pour les exercices de choix, appliquez la méthode du cours : relever les contraintes du cas, les confronter aux critères « quand » de chaque architecture, puis défendre le choix avec la règle (a)–(e). Toutes les solutions sont détaillées en **section 17**, avec la même numérotation.

Exercices faciles

Facile Exercice 16.1

Définissez en une phrase chacune des trois architectures du cours : monolithe, monolithe modulaire et microservices.

Facile Exercice 16.2

Remettez dans l'ordre chronologique les étapes de l'évolution des architectures logicielles : SOA ; client-serveur et monolithes ; cloud-native ; systèmes centralisés ; architecture 3-tiers ; microservices. Associez à chacune sa décennie.

Facile Exercice 16.3

Citez les quatre caractéristiques d'une architecture monolithique données dans le cours.

Facile Exercice 16.4

Vrai ou faux, en justifiant : « Ajouter de la mémoire et des processeurs au serveur unique d'une application, c'est de la scalabilité horizontale. »

Facile **Exercice 16.5**

Donnez l'autre nom (en anglais) de la scalabilité verticale et de la scalabilité horizontale, puis indiquez laquelle des deux présente un point unique de défaillance (SPOF).

Facile **Exercice 16.6**

Citez les six principes des microservices présentés dans le cours.

Facile **Exercice 16.7**

Dans un monolithe modulaire, comment les modules communiquent-ils entre eux ? Et dans une architecture microservices ? Donnez une conséquence de cette différence.

Facile **Exercice 16.8**

Citez les trois scénarios du cours qui motivent l'apparition des microservices.

Facile **Exercice 16.9**

Complétez : « Un monolithe modulaire est composé de modules bien ....., faiblement ..... , mais il reste une ..... unité déployable. Les modules sont organisés par ..... métier et séparés par des ..... internes. »

Facile **Exercice 16.10**

Citez deux avantages et deux limites de l'architecture monolithique.

Exercices moyens

Moyen **Exercice 16.11**

Une banque propose une application web, une application mobile et une application pour guichets automatiques. Chaque canal a été développé avec son propre backend. Identifiez le scénario du cours concerné, citez trois défis qui en découlent et proposez l'organisation recommandée.

Moyen Exercice 16.12

Un site de vente en monolithe tourne sur un serveur de 8 cœurs. Pendant les soldes, seul le module « recherche produits » sature. L'équipe double toute l'application sur un second serveur. Expliquez pourquoi cette solution fonctionne mais gaspille des ressources, et quelle architecture permettrait de ne multiplier que la recherche.

Moyen Exercice 16.13

Une équipe Java/Spring doit intégrer un module de détection de fraude écrit en Python avec des bibliothèques d'apprentissage automatique. Expliquez pourquoi l'intégrer dans le monolithe Java pose problème et décrivez la solution proposée dans le cours.

Moyen Exercice 16.14

Une application de gestion de clinique en monolithe subit une fuite mémoire dans le module « génération de rapports PDF ». Décrivez ce qui arrive aux modules « rendez-vous » et « dossiers patients ». Quelle limite du monolithe cela illustre-t-il ? Ce problème existerait-il dans un monolithe modulaire ?

Moyen Exercice 16.15

Complétez le tableau comparatif de la scalabilité verticale et horizontale selon les quatre critères du cours : implémentation, limite de scalabilité, coût et disponibilité.

Moyen Exercice 16.16

Une start-up de deux développeurs lance une application de réservation de salles de sport. Le produit n'est pas encore validé et le budget d'hébergement est faible. Quelle architecture recommandez-vous ? Justifiez avec au moins trois critères du cours.

Moyen Exercice 16.17

Une équipe de 25 développeurs travaille sur un monolithe non structuré. Chaque correction mineure demande de redéployer toute l'application et les fusions de code bloquent souvent la livraison. Relevez les signes, cités dans le cours, qui montrent que le monolithe devient difficile à gérer.

Moyen **Exercice 16.18**

Un étudiant affirme : « Le monolithe modulaire, c'est déjà des microservices, puisque le code est découpé en modules. » Corrigez cette affirmation en comparant les deux architectures sur trois critères.

Moyen **Exercice 16.19**

Dans une application de gestion d'inventaire, l'opération « valider une vente » doit, en une seule fois, débiter le paiement, décrémenter le stock et créer la facture : soit tout réussit, soit rien. Expliquez pourquoi ce besoin favorise un monolithe ou un monolithe modulaire plutôt que des microservices.

Moyen **Exercice 16.20**

Une plateforme de streaming découpe son système en services « comptes », « catalogue », « lecture vidéo » et « recommandations ». Pour chaque service, indiquez quel principe des microservices il illustre le mieux, puis donnez un inconvénient que ce découpage introduit.

Exercices difficiles

Difficile **Exercice 16.21**

Une librairie de quartier veut vendre en ligne : catalogue de 3 000 livres, panier, paiement, environ 200 commandes par jour, un seul développeur, un hébergement mutualisé et des mises à jour une fois par mois. Choisissez une architecture et défendez-la avec la règle (a)–(e).

Difficile **Exercice 16.22**

Une chaîne de pharmacies régionale (40 officines) emploie 8 développeurs répartis en deux équipes. Ses domaines sont : ordonnances, stock, facturation mutuelles et fidélité. Les règles métier changent souvent, la cohérence ordonnance–stock est critique et l'équipe n'a aucune expérience des systèmes distribués. La direction envisage d'ouvrir un service de livraison dans deux ans. Choisissez et défendez une architecture avec la règle (a)–(e).

Difficile Exercice 16.23

Une application nationale de livraison de repas compte 12 équipes, 2 millions d'utilisateurs, des pics très forts entre 12 h et 14 h sur le suivi des livreurs, un moteur d'estimation des délais en Python, une plateforme Kubernetes et une équipe DevOps dédiée. Choisissez et défendez une architecture avec la règle (a)–(e), puis proposez un découpage en au moins cinq services.

Difficile Exercice 16.24

Un site de billetterie de concerts tourne sur un seul gros serveur. Lors de la mise en vente d'un concert attendu, le trafic est multiplié par 50 pendant une heure, puis redevient normal. L'équipe hésite entre acheter un serveur deux fois plus puissant et passer à plusieurs instances derrière un répartiteur de charge. Comparez les deux options sur les quatre critères du cours et recommandez-en une. Cette décision impose-t-elle de passer aux microservices ?

Difficile Exercice 16.25

Un directeur technique déclare : « Les microservices sont l'architecture moderne ; tout nouveau projet doit les utiliser. » Réfutez cette affirmation en vous appuyant sur l'historique des architectures, sur les critères « quand choisir » de chaque approche et sur les coûts propres aux microservices.

Difficile Exercice 16.26

Une entreprise a migré trop tôt vers 30 microservices avec 6 développeurs. Constat après un an : chaque nouvelle fonctionnalité touche 5 services, les déploiements doivent être synchronisés, les pannes réseau provoquent des erreurs en cascade. Diagnostiquez la situation (quels principes ne sont pas respectés ?) et proposez un plan de correction.

Difficile Exercice 16.27

Une plateforme de e-learning est un monolithe modulaire avec les modules « cours », « vidéos », « quiz », « paiements » et « certificats ». Le module « vidéos » consomme 80 % des ressources et doit être livré chaque semaine, alors que les autres modules changent une fois par trimestre. Proposez une stratégie de migration progressive, en justifiant l'ordre d'extraction et les précautions à prendre.

Difficile Exercice 16.28

Pour un même système de réservation d'hôtels, construisez un tableau comparant monolithe, monolithe modulaire et microservices selon six critères : unité de déploiement, communication interne, scalabilité, isolation des pannes, flexibilité technologique et complexité opérationnelle. Concluez sur le profil d'entreprise adapté à chaque colonne.

Difficile Exercice 16.29

Une mutuelle de taille moyenne (15 développeurs, 3 équipes) gère adhésions, remboursements, documents et relation client. Le module « documents » (numérisation par OCR) connaît des pics en fin de mois et doit utiliser une bibliothèque Python. Le reste exige une forte cohérence et change peu. L'infrastructure est un cloud privé sans orchestrateur. Aucune des trois architectures « pures » ne semble parfaite : proposez une solution argumentée et défendez-la avec la règle (a)–(e).

Difficile Exercice 16.30

Question ouverte du cours : « Comment découper une application en services ? » Pour une application de covoiturage (inscription, publication de trajets, recherche, réservation, paiement, notation, messagerie), proposez un découpage, justifiez chaque frontière par une capacité métier, identifiez deux capacités qu'il vaut mieux garder dans un même service et expliquez pourquoi.

Section 17

Solutions détaillées de la banque d'exercices

Comment lire ces solutions

Chaque solution porte le numéro de l'exercice de la section 16 (17.k corrige 16.k). Cherchez d'abord seul, puis comparez : ce qui compte n'est pas de retrouver mot pour mot la réponse, mais de citer les **bons critères du cours** et de les relier aux **faits de l'énoncé**. Pour les exercices de choix, une autre architecture peut être acceptée si sa défense (a)–(e) couvre toutes les contraintes.

Solutions des exercices faciles

Facile Solution 17.1 — Exercice 16.1

Monolithe : application construite comme **une seule unité**, développée et déployée d'un bloc, dont tous les composants partagent le même environnement d'exécution.

Monolithe modulaire : application composée de **modules bien définis et faiblement couplés**, organisés par capacité métier, mais qui reste **une seule unité déployable**.

Microservices : collection de petits services faiblement couplés et déployables indépendamment, chacun responsable d'une capacité métier et communiquant par API ou messagerie.

La différence essentielle porte donc sur l'*unité de déploiement* : une seule pour les deux premières, une par service pour la troisième.

Facile Solution 17.2 — Exercice 16.2

Ordre	Étape	Décennie
1	Systèmes centralisés	Années 1960
2	Client-serveur et monolithes	Années 1980
3	Architecture 3-tiers	Années 1990
4	SOA (architecture orientée services)	Années 2000
5	Microservices	Années 2010
6	Cloud-native	Années 2020

Chaque étape répond aux limites de la précédente : on découpe progressivement le système en parties de plus en plus indépendantes. Cet ordre ne signifie pas que la dernière étape remplace les autres : le monolithe reste un bon choix dans de nombreux cas.

Facile Solution 17.3 — Exercice 16.3

1. **Unité de déploiement unique** : toute l'application est livrée en une fois.
2. **Environnement d'exécution partagé** : tous les composants tournent dans le même processus.
3. **Composants intégrés via des appels directs** : les modules s'appellent comme des fonctions, en mémoire.
4. **Gestion centralisée** : une seule base de code, une seule configuration, en général une seule base de données.

Ces quatre caractéristiques expliquent à la fois les avantages du monolithe (simplicité, rapidité des appels) et ses limites (tout redéployer, une panne qui se propage).

Facile Solution 17.4 — Exercice 16.4

Faux. Ajouter de la mémoire et des processeurs à un serveur *existant* augmente sa capacité : c'est la **scalabilité verticale** (*scaling up*). La scalabilité horizontale consiste à **ajouter des serveurs ou des instances** supplémentaires de l'application et à répartir la charge entre eux. Ici, il n'y a toujours qu'un seul serveur, donc on reste dans le vertical, avec sa limite matérielle et son point unique de défaillance.

Facile Solution 17.5 — Exercice 16.5

Scalabilité verticale = ***scaling up*** ; scalabilité horizontale = ***scaling out***.

C'est la **scalabilité verticale** qui présente un **point unique de défaillance (SPOF)** : toute l'application repose sur un seul serveur, et si ce serveur tombe, le service s'arrête. L'horizontale, au contraire, apporte de la **redondance** : si une instance tombe, les autres continuent à répondre.

Facile Solution 17.6 — Exercice 16.6

1. **Responsabilité unique** : chaque service gère une seule capacité métier.
2. **Couplage faible** : un service peut changer sans obliger les autres à changer.
3. **Flexibilité technologique** : chaque service peut utiliser le langage et les outils adaptés.
4. **Déploiement indépendant** : un service se livre sans redéployer les autres.
5. **Scalabilité indépendante** : on multiplie uniquement le service qui est sous charge.
6. **Communication par API ou messagerie** : les services échangent par le réseau, via des interfaces bien définies.

Facile Solution 17.7 — Exercice 16.7

Dans un **monolithe modulaire**, les modules communiquent **en mémoire (in-process)**, par des appels de fonctions à travers des interfaces internes. Dans une architecture **microservices**, ils communiquent **par le réseau**, via des API (HTTP) ou une messagerie.

Conséquence : un appel in-process est rapide et ne peut pas « se perdre » ; un appel réseau est plus lent et peut échouer (service indisponible, délai dépassé). Les microservices doivent donc prévoir la gestion de ces pannes, ce que le monolithe modulaire n'a pas à faire. En contrepartie, le monolithe modulaire ne peut pas déployer ni multiplier un module seul.

Facile Solution 17.8 — Exercice 16.8

1. **Multi-plateformes** : plusieurs clients (web, mobile…) qui doivent partager un même backend exposé par une API.
2. **Hétérogénéité technologique** : une partie du système doit utiliser une autre technologie (par exemple de l'IA en Python à côté d'une application Java/Spring).
3. **Scalabilité indépendante** : une seule partie subit la charge (par exemple le catalogue pendant le Black Friday).

Dans les trois cas, le monolithe oblige à traiter l'application comme un bloc, alors que le besoin ne concerne qu'une partie.

Facile Solution 17.9 — Exercice 16.9

« Un monolithe modulaire est composé de modules bien **définis**, faiblement **couplés**, mais il reste une **seule** unité déployable. Les modules sont organisés par **capacité** métier et séparés par des **interfaces** internes. »

Ces cinq mots résument la notion : la structure interne ressemble à des services (frontières claires), mais le déploiement reste celui d'un monolithe.

Facile Solution 17.10 — Exercice 16.10

Avantages (deux parmi quatre) : développement simple (un seul projet, un seul langage, un seul environnement) ; déploiement facile (un seul artefact) ; tests simplifiés (tout se lance ensemble) ; communication efficace (appels directs en mémoire).

Limites (deux parmi quatre) : scalabilité limitée (on multiplie tout ou rien) ; complexité croissante quand le code grossit ; dépendances de déploiement (une petite modification impose de tout redéployer) ; propagation des pannes (un composant défaillant peut arrêter toute l'application).

Solutions des exercices moyens

Moyen Solution 17.11 — Exercice 16.11

Scénario : c'est le scénario **multi-plateformes** de N1 : trois canaux (web, mobile, guichets) pour un même métier.

Défis : ① *duplication* : les mêmes règles (solde, virement, plafond) sont codées trois fois ; ② *maintenance* : chaque évolution doit être faite et testée trois fois ; ③ *incohérence* : un canal peut appliquer une règle que l'autre a oubliée (un plafond différent au guichet et sur mobile) ; on peut ajouter les *coûts* de trois équipes et trois infrastructures.

Organisation recommandée : un seul backend qui contient la logique métier et l'expose par une **API** ; les trois clients deviennent de simples interfaces qui appellent cette API. Une règle n'est alors écrite qu'une fois et s'applique à tous les canaux.

Moyen Solution 17.12 — Exercice 16.12

Pourquoi cela fonctionne : doubler l'application est de la **scalabilité horizontale** ; avec un répartiteur de charge, les requêtes de recherche sont partagées entre deux copies, donc la recherche dispose de deux fois plus de ressources.

Pourquoi c'est du gaspillage : dans un monolithe, on ne peut copier que *toute* l'application. Le second serveur héberge aussi le panier, les comptes, les paiements… qui n'étaient pas saturés. On paie donc des ressources (mémoire, CPU, licences) pour des modules qui n'en avaient pas besoin : c'est exactement le défi « gaspillage des ressources et coûts » du scénario de la scalabilité indépendante (N1).

Architecture adaptée : les **microservices**, dont le principe de **scalabilité indépendante** permet de lancer plusieurs instances du seul service « recherche » pendant les soldes, puis de les retirer ensuite.

Moyen Solution 17.13 — Exercice 16.13

Problème : un monolithe partage **un seul environnement d'exécution** (ici la JVM). Le code Python et ses bibliothèques d'apprentissage automatique ne s'y exécutent pas directement : c'est l'*incompatibilité de langage*. Les contourner (réécrire en Java, appeler Python depuis Java) crée des difficultés d'*intégration*, des *dépendances de déploiement* (chaque mise à jour du modèle oblige à redéployer tout le monolithe) et nuit à la *maintenabilité*.

Solution du cours : faire de la détection de fraude un **service séparé**, écrit en Python, qui expose une API ; l'application Java l'appelle par **HTTP**. Chaque partie garde sa technologie et se déploie à son rythme : c'est le principe de **flexibilité technologique**.

Moyen Solution 17.14 — Exercice 16.14

Ce qui arrive : tous les modules tournent dans le **même processus** et partagent la même mémoire. La fuite du module « rapports PDF » consomme peu à peu toute la mémoire, puis le processus ralentit et finit par planter : « rendez-vous » et « dossiers patients » deviennent indisponibles alors qu'ils n'ont aucun défaut.

Limite illustrée : la **propagation des pannes**, conséquence de l'environnement d'exécution partagé.

Dans un monolithe modulaire ? Oui, le problème existerait toujours. Les modules y sont séparés dans le *code*, mais ils s'exécutent encore dans un seul processus (communication in-process, déploiement unique). Seuls des services déployés séparément, chacun dans son propre processus, isoleraient la panne.

Moyen Solution 17.15 — Exercice 16.15

Critère	Verticale (scaling up)	Horizontale (scaling out)
Implémentation	Simple : on améliore le serveur existant	Configuration supplémentaire : répartiteur de charge, coordination
Limite de scalabilité	Limitée par la capacité du matériel	Extensible en ajoutant des serveurs
Coût	Le matériel puissant peut coûter cher	Plusieurs serveurs abordables
Disponibilité	Point unique de défaillance (SPOF)	Redondance : meilleure disponibilité

Lecture du tableau : la verticale gagne sur la simplicité, l'horizontale sur la croissance et la disponibilité. Le choix dépend des exigences de performance, des coûts et de la complexité acceptable.

Moyen Solution 17.16 — Exercice 16.16

Recommandation : un monolithe (avec un code rangé par domaine).

Justification par les critères « quand choisir » du monolithe : ① *petite équipe* : deux développeurs ne peuvent pas maintenir plusieurs services, déploiements et bases ; ② *projet précoce* : le produit n'est pas validé, il faut pouvoir changer vite le fonctionnement, ce qui est plus simple dans une seule base de code ; ③ *infrastructure limitée* : un budget faible correspond à un seul serveur et une seule base ; ④ *petite application* : réservation de salles, quelques fonctionnalités seulement.

Les microservices ajouteraient des coûts (réseau, supervision, plusieurs déploiements) sans aucun besoin de scalabilité ou de technologie indépendantes. Si le produit réussit et l'équipe grandit, on pourra évoluer vers un monolithe modulaire.

Moyen Solution 17.17 — Exercice 16.17

Le cours cite cinq signes ; l'énoncé en montre au moins trois :

- **Redéploiement** : « chaque correction mineure demande de redéployer toute l'application » — dépendances de déploiement.
- **Coordination des équipes** : 25 développeurs sur une même base de code ; « les fusions de code bloquent la livraison ».
- **Complexité** : un monolithe « non structuré » de cette taille devient difficile à comprendre et à modifier sans effet de bord.

Les deux autres signes (*scalabilité* et *contraintes technologiques*) ne sont pas mentionnés ici. Une première réponse adaptée serait de structurer le code en modules (monolithe modulaire) avant d'envisager des microservices.

Moyen Solution 17.18 — Exercice 16.18

L'affirmation est **fausse** : découper le *code* ne suffit pas, ce qui définit les microservices est le découpage du *déploiement* et de l'*exécution*.

- **Déploiement :** monolithe modulaire = une seule unité déployable ; microservices = chaque service se déploie indépendamment.
- **Communication :** monolithe modulaire = appels en mémoire (in-process) ; microservices = appels réseau (API, messagerie).
- **Scalabilité :** monolithe modulaire = on multiplie toute l'application ; microservices = on multiplie un seul service.

On peut ajouter la technologie : un seul langage d'exécution dans le monolithe modulaire, un langage par service possible en microservices. Le monolithe modulaire est donc une étape intermédiaire qui *prépare* une migration, pas une architecture microservices.

Moyen **Solution 17.19 — Exercice 16.19**

Le besoin « soit tout réussit, soit rien » est celui d'une **transaction atomique**. Dans un monolithe (ou un monolithe modulaire), paiement, stock et facture sont dans la même application et la même base : une seule transaction englobe les trois écritures, et la base annule tout automatiquement en cas d'échec.

En microservices, chaque service possède ses propres données : paiement, stock et facture seraient trois bases différentes reliées par le réseau. Il faudrait coordonner trois opérations distantes, prévoir les cas où l'une échoue après que les autres ont réussi (paiement débité mais stock non décrémenté) et écrire du code d'annulation. C'est pourquoi le cours range les « transactions atomiques fréquentes » et la « forte cohérence » parmi les critères qui favorisent le monolithe et le monolithe modulaire.

Moyen **Solution 17.20 — Exercice 16.20**

- **Comptes** → *responsabilité unique* : il ne gère qu'une capacité, l'identité et l'abonnement des utilisateurs.
- **Catalogue** → *couplage faible / déploiement indépendant* : on peut ajouter des fonctions de navigation et le livrer sans toucher à la lecture vidéo.
- **Lecture vidéo** → *scalabilité indépendante* : c'est lui qui subit la charge le soir ; on multiplie ses instances seules.
- **Recommandations** → *flexibilité technologique* : il peut être écrit en Python avec des bibliothèques d'IA.

Inconvénient introduit : les services communiquent désormais **par le réseau** ; un appel peut échouer ou être lent (si « recommandations » ne répond pas, la page d'accueil doit fonctionner quand même), et il faut déployer, superviser et maintenir quatre services au lieu d'un : la complexité opérationnelle augmente.

Solutions des exercices difficiles

Difficile **Solution 17.21 — Exercice 16.21**

(a) Architecture : monolithe, avec un code rangé par domaine (catalogue, panier, paiement).

(b) Trois faits : ① *un seul développeur* : il ne peut pas maintenir plusieurs services ; ② *200 commandes par jour* et 3 000 livres : une charge faible, sans pic propre à un module, qu'un seul

serveur absorbe ; ③ *hébergement mutualisé* : il ne permet de déployer qu'une application simple, sans orchestrateur.

(c) Pourquoi pas les autres : les *microservices* exigent une infrastructure mature et une équipe capable de les exploiter, et casseraient la transaction panier–paiement–stock pour aucun bénéfice. Le *monolithe modulaire* répond à des problèmes de coordination entre équipes qui n'existent pas avec un seul développeur ; ses frontières strictes seraient un effort sans retour.

(d) Inconvénient : toute modification impose de redéployer le site entier, et une panne d'un composant (par exemple le paiement) peut rendre tout le site indisponible.

(e) Déclencheur : l'arrivée de plusieurs développeurs qui se gênent dans le code, ou une forte croissance des ventes (passage à une boutique multi-magasins) → passer au monolithe modulaire.

Cette situation est proche du **Cas 1** de la section 15 ; les mises à jour mensuelles confirment qu'aucun déploiement indépendant n'est nécessaire.

Difficile Solution 17.22 — Exercice 16.22

(a) Architecture : monolithe modulaire : quatre modules (ordonnances, stock, facturation mutuelles, fidélité), chacun avec ses interfaces internes, déployés ensemble.

(b) Trois faits : ① *deux équipes de 8 développeurs* : des frontières claires entre modules évitent qu'elles se gênent ; ② *cohérence ordonnance–stock critique* : la communication in-process et la base commune permettent une transaction atomique ; ③ *aucune expérience des systèmes distribués* : les microservices seraient un risque majeur pour une activité de santé.

(c) Pourquoi pas les autres : un *monolithe* non structuré ne protégerait pas les deux équipes l'une de l'autre et supporterait mal des « règles métier qui changent souvent » (effets de bord entre domaines). Les *microservices* casseraient la cohérence ordonnance–stock, demanderaient une infrastructure mature, et 40 officines ne génèrent pas de besoin de scalabilité indépendante.

(d) Inconvénient : une seule unité déployable : une correction de la fidélité oblige à redéployer aussi les ordonnances, et une panne d'un module peut affecter les autres.

(e) Déclencheur : l'ouverture du **service de livraison**, s'il apporte une charge ou un rythme de livraison très différents (suivi en temps réel), ou si l'équipe acquiert l'expérience des systèmes distribués → extraire ce module en service séparé. Le monolithe modulaire prépare justement cette « migration future ».

Difficile Solution 17.23 — Exercice 16.23

(a) Architecture : microservices.

(b) Trois faits : ① *12 équipes* : chacune a besoin d'autonomie et de déployer sans attendre les autres ; ② *pics de 12 h à 14 h sur le suivi des livreurs* : une partie seulement du système doit être multipliée (scalabilité indépendante) ; ③ *moteur d'estimation en Python, Kubernetes et équipe DevOps* : flexibilité technologique nécessaire et infrastructure mature disponible.

(c) Pourquoi pas les autres : un *monolithe* obligerait à multiplier toute l'application pour le seul suivi des livreurs, et 12 équipes dans une base de code créeraient des blocages de livraison. Un *monolithe modulaire* clarifierait le code, mais garderait un déploiement unique pour 12 équipes, une scalabilité globale et un seul environnement d'exécution (problème du moteur Python).

(d) Inconvénient : forte complexité opérationnelle (réseau, supervision, versions d'API) et cohérence des données plus difficile entre commandes et paiements, qui ne sont plus dans la même base.

(e) Déclencheur : si les services deviennent trop couplés (une fonctionnalité touche systématiquement plusieurs services) ou si les coûts d'exploitation dépassent les gains → regrouper certains services.

Découpage proposé (sept services, un par capacité métier) :

Service	Capacité métier	Remarque
Comptes	Inscription, connexion, profils clients et livreurs	Sollicité par tous les autres
Restaurants & menus	Catalogue des restaurants et des plats	Lecture intensive
Commandes	Création et cycle de vie d'une commande	Cœur du métier
Paiements	Encaissement, remboursements	Exigences de fiabilité fortes
Suivi des livreurs	Positions en temps réel	Mis à l'échelle seul de 12 h à 14 h
Estimation des délais	Calcul du temps de livraison	Écrit en Python
Notifications	SMS, push, e-mails	Alimenté par messagerie

Difficile Solution 17.24 — Exercice 16.24

Critère	Serveur deux fois plus puissant (vertical)	Plusieurs instances + répartiteur (horizontal)
Implémentation	Simple : on remplace le serveur, aucun changement de configuration	Il faut un répartiteur de charge et partager l'état (sessions, réservations) entre instances
Limite de scalabilité	Un serveur ×2 ne suffit pas pour un trafic ×50 ; la limite matérielle est vite atteinte	On ajoute autant d'instances que nécessaire
Coût	Matériel cher, payé toute l'année pour une heure de pic	Serveurs abordables, ajoutés pour le pic puis retirés
Disponibilité	SPOF : si le serveur tombe au moment de la mise en vente, tout s'arrête	Redondance : la perte d'une instance n'arrête pas la vente

Recommandation : la scalabilité horizontale. Le pic est énorme (×50), court et ponctuel : seul l'ajout temporaire d'instances suit cette forme de charge sans payer en permanence un matériel surdimensionné, et la redondance protège le moment le plus critique pour le chiffre d'affaires. Le prix à payer est la configuration (répartiteur, état partagé), notamment pour ne pas vendre deux fois la même place.

Faut-il passer aux microservices ? Non. Le cours le précise : la scalabilité horizontale consiste à déployer **plusieurs copies de la même application monolithique**. Les microservices ne deviendraient utiles que si une seule partie (par exemple la file d'attente d'achat) devait être multipliée séparément ou si l'organisation des équipes l'exigeait.

Difficile Solution 17.25 — Exercice 16.25

1. L'historique ne désigne pas un « gagnant ». Des systèmes centralisés (1960) au cloud-native (2020), chaque étape répond aux limites de la précédente *dans un contexte donné* (grandes organisations, forte charge, nombreux canaux). Les microservices sont apparus pour des entreprises qui avaient ces problèmes ; ce n'est pas une évolution obligatoire pour toutes.

2. Chaque architecture a ses critères « quand ». Le monolithe convient aux petites applications, petites équipes, projets précoces, infrastructures limitées et transactions atomiques fréquentes. Le monolithe modulaire convient aux applications moyennes, aux exigences

évolutives et à la forte cohérence. Les microservices ne se justifient qu'avec de grandes applications, des équipes autonomes, des besoins de scalabilité ou de technologie indépendantes *et* une infrastructure mature. Un nouveau projet est presque toujours petit et incertain : il coche les critères du monolithe.

3. Les microservices ont un coût propre. Communication réseau (lenteur, pannes partielles, erreurs en cascade), données réparties (perte des transactions atomiques), plusieurs déploiements et une supervision à mettre en place, compétences DevOps. Payer ces coûts sans en avoir les bénéfices ralentit l'équipe.

Conclusion : l'architecture se choisit selon les contraintes du projet, pas selon la mode. Le LAB le rappelle : « les microservices ne sont pas supérieurs par nature ». Une approche raisonnable est de commencer par un monolithe (modulaire) et d'extraire des services quand un besoin réel apparaît.

Difficile Solution 17.26 — Exercice 16.26

Diagnostic. L'entreprise a obtenu les coûts des microservices sans leurs bénéfices ; on parle souvent de « monolithe distribué ». Principes non respectés :

- **Responsabilité unique / découpage par capacité métier** : si une fonctionnalité touche 5 services, les frontières suivent des couches techniques ou des morceaux trop petits, pas des capacités métier complètes.
- **Couplage faible** : les services dépendent fortement les uns des autres.
- **Déploiement indépendant** : des déploiements « synchronisés » montrent qu'aucun service ne peut évoluer seul.
- **Condition d'infrastructure mature et d'équipe suffisante** : 6 développeurs pour 30 services, soit 5 services par personne ; les pannes réseau en cascade montrent que la communication n'est pas maîtrisée.

Plan de correction.

1. **Cartographier** les capacités métier réelles et les dépendances entre services (quels services changent toujours ensemble ?).
2. **Regrouper** les services qui changent ensemble : revenir à quelques unités (par exemple un monolithe modulaire avec un module par capacité métier), ce qui rétablit les appels in-process et les transactions.
3. **Ne garder séparés** que les services qui ont un vrai besoin : charge propre, technologie différente ou rythme de livraison distinct.

4. **Fiabiliser** les appels réseau restants (délais d'attente, reprises, fonctionnement dégradé) et mettre en place une supervision.
5. **Revoir le choix** plus tard, quand l'équipe et l'infrastructure auront grandi.

Difficile Solution 17.27 — Exercice 16.27

Stratégie : migration progressive, un module à la fois, en gardant le reste en monolithe modulaire.

1. Extraire d'abord « vidéos ». Il cumule les deux justifications des microservices : *scalabilité indépendante* (80 % des ressources : on veut multiplier ce seul module) et *déploiement indépendant* (livré chaque semaine, alors que le reste change chaque trimestre). C'est aussi le module dont l'extraction rapporte le plus pour un seul effort.

2. Garder dans le monolithe modulaire « cours », « quiz », « paiements » et « certificats » : ils changent rarement, n'ont pas de charge particulière, et « paiements »–« certificats » demandent de la cohérence (un certificat n'est délivré qu'après paiement). Les extraire n'apporterait que des coûts. On ne les extrairait plus tard que si un besoin réel apparaissait.

Précautions :

- Partir de l'**interface interne existante** du module « vidéos » pour définir son **API** : la frontière est déjà claire, c'est l'avantage du monolithe modulaire.
- Donner au service ses **propres données** (métadonnées et fichiers vidéo) et supprimer tout accès direct depuis les autres modules.
- Prévoir les **pannes réseau** : si le service vidéo ne répond pas, le reste de la plateforme (quiz, paiements) doit continuer à fonctionner.
- Mettre en place **déploiement automatisé et supervision** avant la bascule, et basculer progressivement en gardant un retour arrière possible.

Difficile Solution 17.28 — Exercice 16.28

Critère	Monolithe	Monolithe modulaire	Microservices
Unité de déploiement	Une seule	Une seule	Une par service (réservations, chambres, paiements…)

Critère	Monolithe	Monolithe modulaire	Microservices
Communication interne	Appels directs en mémoire	In-process, via des interfaces internes	Réseau : API ou messagerie
Scalabilité	Toute l'application (verticale ou copies)	Toute l'application	Par service (ex. recherche de disponibilités seule)
Isolation des pannes	Faible : une panne peut tout arrêter	Faible : même processus	Forte : une panne reste dans son service
Flexibilité technologique	Un seul langage	Un seul langage	Un langage par service possible
Complexité opérationnelle	Faible	Faible à moyenne	Élevée (déploiements, réseau, supervision)

Conclusion par profil :

- **Monolithe** : un hôtel indépendant ou une petite chaîne, petite équipe, peu de trafic, réservation et paiement à garder dans une même transaction.
- **Monolithe modulaire** : une chaîne régionale avec plusieurs équipes moyennes, des règles qui évoluent (tarifs, fidélité) et un besoin de forte cohérence, sans expertise des systèmes distribués.
- **Microservices** : une grande plateforme de réservation nationale ou internationale, avec de nombreuses équipes autonomes, une recherche soumise à de très forts pics et une infrastructure mature.

Difficile Solution 17.29 — Exercice 16.29

Proposition : une **solution mixte** — un **monolithe modulaire** (adhésions, remboursements, relation client) et **un seul service séparé « documents »** écrit en Python, appelé par API ou alimenté par une file de messages.

(a) Architecture : monolithe modulaire + un service extrait (documents/OCR).

(b) Trois faits : ① *3 équipes, 15 développeurs, forte cohérence et peu de changements* pour adhésions, remboursements et relation client → critères du monolithe modulaire ; ② *bibliothèque Python* pour l'OCR → flexibilité technologique, impossible dans l'environnement d'exécution du monolithe ; ③ *pics en fin de mois* limités aux documents → scalabilité indépendante de ce seul module.

(c) Pourquoi pas les autres : un *monolithe* (même modulaire) *pur* ne peut ni exécuter la bibliothèque Python ni multiplier l'OCR seul : il faudrait copier toute l'application en fin de mois. Des *microservices complets* casseraient la cohérence adhésions–remboursements et demanderaient un orchestrateur et une expertise absents ; un seul service supplémentaire, lui, reste gérable sur quelques machines du cloud privé derrière un répartiteur.

(d) Inconvénient : deux modes de déploiement et une communication réseau à maîtriser ; si le service documents est indisponible, les dossiers en attente d'OCR sont retardés (d'où l'intérêt d'une file de messages, qui garde les demandes jusqu'à ce que le service revienne).

(e) Déclencheur : si d'autres modules développent à leur tour des besoins de charge ou de technologie propres, et que l'entreprise se dote d'un orchestrateur et d'une équipe d'exploitation → extraire d'autres services ; à l'inverse, si l'OCR devient disponible en Java et les pics disparaissent → réintégrer le module.

Difficile Solution 17.30 — Exercice 16.30

Principe : une frontière de service = une **capacité métier** complète, avec ses propres données, qui peut évoluer sans obliger les autres à changer (responsabilité unique, couplage faible).

Service	Capacité métier et justification de la frontière
Utilisateurs	Inscription, profils, vérification d'identité : données personnelles gérées à un seul endroit.
Trajets & réservations	Publication d'un trajet et réservation des places : gèrent ensemble le nombre de places disponibles.
Recherche	Trouver un trajet par ville et date : lecture seule, très sollicitée (vendredis, vacances), multipliée seule.
Paiement	Encaissement, reversement au conducteur, remboursements : exigences de sécurité propres.
Notation	Avis après le trajet : capacité indépendante, rythme d'évolution propre.
Messagerie	Échanges conducteur–passagers : charge et technologie (temps réel) particulières.

Capacités à garder dans un même service : publication de trajets et réservation. Réserver une place modifie le nombre de places du trajet publié ; si deux services différents détenaient ces données, deux passagers pourraient réserver la dernière place en même temps (surréservation).

Dans un même service, la décrémentation des places et la création de la réservation tiennent dans **une transaction atomique**. Séparer ces deux capacités créerait un couplage fort : presque toute évolution de l'une toucherait l'autre.

À l'inverse, la recherche peut être séparée : elle ne fait que *lire* une copie des trajets, et la recevoir avec quelques secondes de retard est acceptable. C'est la réponse à la question ouverte du cours : on découpe selon les capacités métier, mais on garde ensemble ce qui doit rester cohérent.